ECOLE MAROCAINE DES SCIENCES DE L'INGENIEUR
Membre de HONORIS UNITED UNIVERSITIES

ANNÉE UNIVERSITAIRE 2025/2026

RAPPORT DE PROJET

Fiducia: Portail de suivi du doctorat

4 ème Année Ingénierie Informatique et Réseaux

21 Mai 2026

RÉALISÉ PAR

AIT LAHCEN Achraf
EZ-ZAHRAOUI El-Mehdi
LAHLOU Chaimaa

PROFESSEUR

Mme AHAIDOUS Khadija

Dédicaces

À ceux qui ont éclairé notre chemin vers la connaissance, on dédie ce travail avec une profonde reconnaissance et une gratitude sincère.

À nos chers parents, pilier inébranlable de notre vie, dont l'amour, le soutien moral et les sacrifices silencieux nous ont porté tout au long de ce parcours, nous donnant la force de surmonter chaque difficulté et de croire en nos capacités.

À nos enseignants, pour leur patience infinie, leurs conseils éclairés et leurs remarques constructives, qui ont su guider nos pas, stimuler notre réflexion et nourrir notre désir d'apprendre, transformant cette expérience en une véritable aventure intellectuelle.

À vous, cher lecteur, qui prenez le temps de parcourir ces pages, recevez ici toute notre reconnaissance, en espérant que ce travail saura vous apporter autant qu'il nous a apporté lors de sa rédaction.

Remerciements

Avant d'entamer ce rapport, on souhaite profiter de cette occasion pour exprimer nos profonds remerciements à toutes les personnes qui ont contribué, de près ou de loin, à sa réalisation et à son élaboration.

On tient à remercier infiniment notre enseignante en Architecture JEE Mme AHAIDOUS Khadija, pour son temps, sa bienveillance et ses précieux conseils. Je remercie également l'ensemble des membres de l'École Marocaine des Sciences de l'Ingénieur, en particulier M. ERRAJI Naoufal, Responsable de la filière, et Mme BADRITIJANE Fatimazahra, chef de département, pour le soutien et l'accompagnement dont ils nous ont fait bénéficier.

Enfin, on tient à adresser nos profonds et sincères remerciements à tous nos enseignants pour leur encadrement et leur dévouement.

Table des matières

Introduction générale 1
Chapitre 1 : Présentation du cadre du projet 3
1. Introduction 4
2. Présentation du cadre du projet 4
3. Étude de l'existant 4
3.1 Description de l'existant 4
3.2 Critique de l'existant 5
3.3 Solution proposée 5
4. Choix du modèle de développement 5
5. Planning prévisionnel 6
6. Conclusion 6
Chapitre 2 : Spécifications des besoins 8
1. Introduction 9
2. Spécification des besoins fonctionnels 9
2.1 Besoin fonctionnel 1 : Gestion des comptes et authentification 9
2.2 Besoin fonctionnel 2 : Gestion du processus d'inscription et de réinscription 10
2.3 Besoin fonctionnel 3 : Gestion du processus de soutenance 10
2.4 Besoin fonctionnel 4 : Suivi du dossier et visibilité sur l'avancement 10
3. Spécification des besoins non fonctionnels 11
4. Présentation des cas d'utilisation 11
4.1 Présentation des acteurs 11
4.2 Description des cas d'utilisation 12
4.3 Diagramme global des cas d'utilisation 12
Chapitre 3 : Conception du système 14
1. Introduction 15
2. Modélisation dynamique 15
2.1. Diagramme de séquence 15
2.2 Diagramme d'activité 17
3. Modélisation statique 19
3.1 Diagramme de classes 20
3.2 Modèle relationnel 22
3.3 Dictionnaire de données 23
4. Architecture de l'application 25
4.1 Architecture logicielle 25
4.2 Architecture matérielle 26

5. Conclusion 27
Chapitre 4 : Réalisation du système 28
1. Introduction 29
2. Environnement de développement 29
2.1 Environnement matériel 29
2.2 Environnement logiciel 29
3. Principales interfaces graphiques 31
4. Conclusion 34
Chapitre 5 : Tests et validations 35
1. Introduction 36
2. Objectifs des tests 36
3. Environnement de test 37
4. Méthodologie de test 37
5. Cas de test fonctionnels 38
5.1. Test d'inscription d'un candidat 38
5.2. Test d'inscription d'un directeur de thèse 38
5.3. Test d'inscription d'un administrateur 39
5.4. Test de connexion 40
5.5. Test de consultation des utilisateurs 40
5.6. Test d'ajout d'un rôle à un utilisateur 40
5.7. Test de suppression d'un rôle 41
5.8. Test de création d'une campagne d'inscription 41
5.9. Test de consultation des campagnes 42
5.10. Test de récupération de la campagne active 42
5.11. Test de désactivation d'une campagne 42
5.12. Test de soumission d'un dossier candidat 43
5.13. Test de consultation du statut du candidat 43
5.14. Test de suivi détaillé du dossier 43
5.15. Test d'avis du directeur 44
5.16. Test de finalisation administrative 44
5.17. Test de consultation du profil utilisateur 44
5.18. Test de mise à jour du profil 45
5.19. Test de consultation des dossiers du directeur 45
6. Synthèse des validations 45
7. Conclusion 46
Conclusion générale 47

Bibliographie 49
Annexe A : Autres Interfaces Graphiques 50

Liste des figures

Figure 1 : Planning prévisionnel 6
Figure 2 : Diagramme de cas d'utilisation 13
Figure 3 : Diagramme de séquence 16
Figure 4 : Diagramme d'activité 18
Figure 5 : Diagramme de classes 20
Figure 6 : Diagramme de déploiement 27
Figure 7 : Git 30
Figure 8 : Java 30
Figure 9 : Docker 30
Figure 10 : Spring boot 30
Figure 11 : Postman 31
Figure 12 : Thymeleaf 31
Figure 13 : Maven 31
Figure 14 : Postgresql 31
Figure 15 : Spring Security 31
Figure 16 : Spring data JPA 31
Figure 17 : Page login 32
Figure 18 : Page Sign in 32
Figure 19 : Dashboard Doctorant 33
Figure 20 : Dossier d'inscription 33
Figure 21 : Demande de soutenance 34
Figure 22 : Profil doctorant 50
Figure 23 : Dashboard directeur gestion des candidatures 50
Figure 24 : Evaluation directeur 51
Figure 25 : Proposition jury par directeur 51
Figure 26 : Profil directeur 52
Figure 27 : Gestion des utilisateurs 52
Figure 28 : Validation dossier par admin 53
Figure 29 : Décision admin 53
Figure 30 : Planification de soutenance par admin 54
Figure 31 : Gestion des campagnes d'inscription 54
Figure 32 : Inscription des directeurs 55
Figure 33 : Inscription administrative 55
Figure 34 : Profil administration 56
Figure 35 : Résultat soutenance 56

Liste des tableaux

Tableau 1 : Planning prévisionnel 6
Tableau 2 : Acteurs 12
Tableau 3 : Dictionnaire de données 25

Introduction générale

Fiducia est une application web de suivi du parcours doctoral conçue pour centraliser et automatiser l'ensemble des démarches liées à l'inscription, la réinscription annuelle et la demande de soutenance. Elle répond à un besoin réel de modernisation d'un processus encore largement fondé sur des échanges manuels, des documents papier et des validations dispersées, ce qui entraîne souvent des retards, des erreurs et un manque de visibilité pour les différents acteurs du système. Le projet vise ainsi à offrir une plateforme unique, moderne et efficace, permettant de simplifier la gestion administrative du doctorat tout en améliorant la coordination entre doctorants, directeurs de thèse et personnel administratif.

D'un point de vue fonctionnel, Fiducia organise le suivi du doctorant autour de plusieurs modules essentiels : la gestion des comptes et de l'authentification, le processus d'inscription et de réinscription, ainsi que le processus de soutenance. Chaque utilisateur dispose d'un rôle clairement défini, avec des permissions adaptées à ses responsabilités : le doctorant soumet ses dossiers et suit leur avancement, le directeur de thèse consulte le statut du doctorant et valide certaines étapes, tandis que l'administration contrôle la conformité des dossiers et supervise le processus global. Cette structuration permet d'assurer un circuit de validation fluide, traçable et sécurisé.

Le projet met également l'accent sur le respect des règles de gestion académiques, notamment les délais de réinscription, la durée maximale du doctorat et les conditions préalables à la soutenance. Le système doit vérifier automatiquement plusieurs critères, comme le nombre de publications, les heures de formation doctorale ou encore la présence des documents exigés pour la soutenance. L'objectif est de garantir un meilleur suivi des dossiers, une prise de décision plus rapide et une réduction des interventions manuelles dans le traitement administratif.

Sur le plan technique, Fiducia s'appuie sur une architecture web moderne utilisant Spring Boot 3, Spring MVC, Spring Data JPA et Spring Security pour le backend, avec une base de données PostgreSQL et H2 pour le développement local. Le rendu côté serveur est assuré par Thymeleaf, tandis que les outils de développement et de validation incluent IntelliJ IDEA, Postman, Git, Maven, ainsi qu'un déploiement possible via Docker. Ce choix technologique favorise la robustesse, la sécurité, la maintenabilité et l'évolutivité de l'application.

Le projet Fiducia illustre ainsi une démarche complète de digitalisation du suivi doctoral, en combinant centralisation des données, automatisation des validations et amélioration de l'expérience utilisateur. Il constitue une réponse concrète aux besoins des établissements d'enseignement supérieur souhaitant moderniser la gestion administrative du doctorat et offrir un meilleur accompagnement aux doctorants tout au long de leur parcours.

- Le chapitre 1 pose le cadre du projet en présentant le contexte général, les objectifs visés et les choix technologiques retenus pour sa mise en œuvre.
- Le chapitre 2 décrit les spécifications des besoins, en détaillant les exigences fonctionnelles et non fonctionnelles, ainsi que les besoins propres à chaque composant de l'application.
- Le chapitre 3 explique la conception du système, en présentant l'architecture logicielle, les modèles de données et les choix de conception adoptés.
- Le chapitre 4 traite de la réalisation du système, en détaillant les aspects techniques liés au développement, à la configuration, au déploiement et aux tests de validation.
- Le chapitre 5 est consacré aux tests et à la validation du système afin de vérifier sa conformité par rapport au cahier des charges.

PRÉSENTATION DU CADRE DU PROJET

01

1. Introduction

Ce premier chapitre pose les bases du projet en situant son contexte général et en justifiant la nécessité de développer une plateforme web de suivi du doctorat. Le processus actuel de gestion des inscriptions, des réinscriptions et des demandes de soutenance repose encore sur des échanges manuels, des documents papier et des validations dispersées, ce qui engendre des retards, des erreurs et un manque de visibilité pour les différents acteurs concernés. L'étude du cadre du projet permet ainsi de mettre en évidence les limites de l'existant et de souligner les besoins précis auxquels Fiducia doit répondre. Ce chapitre présente également les objectifs du projet, la solution proposée, ainsi que l'approche retenue pour sa réalisation. Enfin, il introduit les grandes étapes du travail et le planning prévisionnel de développement, afin de donner une vision claire du déroulement global du projet.

2. Présentation du cadre du projet

Le projet Fiducia s'inscrit dans le cadre de la digitalisation des processus administratifs liés au parcours doctoral. Il a pour objectif de concevoir et de développer une application web centralisée permettant de gérer de manière efficace l'inscription, la réinscription annuelle et la soutenance des doctorants. Cette plateforme vise à améliorer la coordination entre les doctorants, les directeurs de thèse et le personnel administratif, tout en assurant un meilleur suivi des dossiers et des règles de gestion académiques.

Dans un contexte universitaire où la gestion du doctorat implique de nombreuses étapes administratives, la mise en place d'un système d'information dédié devient nécessaire. Fiducia répond à ce besoin en offrant un environnement structuré, sécurisé et accessible, capable de centraliser les données et documents relatifs à chaque doctorant. Le projet s'inscrit ainsi dans une démarche de modernisation des services administratifs et d'optimisation du suivi des parcours doctoraux.

3. Étude de l'existant

3.1 Description de l'existant

Actuellement, le suivi du doctorat repose dans de nombreux cas sur des procédures manuelles ou semi-manuelles. Les doctorants soumettent leurs dossiers par voie physique ou par courriel, tandis que les validations successives sont effectuées séparément par les encadrants et l'administration. Les documents requis pour l'inscription, la réinscription ou la soutenance sont souvent dispersés entre plusieurs supports, ce qui complique leur traitement et leur archivage.

Cette organisation ne permet pas une vision globale et en temps réel de l'état d'avancement des dossiers. Chaque acteur intervient à un moment précis du processus, mais sans véritable centralisation des informations ni automatisation du circuit de validation. Il en résulte une gestion lente, peu fluide et difficile à suivre, notamment lorsque plusieurs dossiers doivent être traités simultanément.

3.2 Critique de l'existant

Le système actuel présente plusieurs limites. D'abord, il est fortement dépendant des interventions humaines, ce qui augmente le risque d'erreurs, d'oublis et de pertes d'information. Ensuite, il est chronophage, car chaque dossier doit être vérifié, transmis et validé manuellement par différents intervenants. De plus, l'absence de centralisation rend difficile le suivi des documents, la consultation de l'historique des actions et le contrôle du respect des conditions de soutenance.

Par ailleurs, les règles de gestion académiques, comme la durée maximale du doctorat ou les prérequis à la soutenance, ne sont pas toujours vérifiées automatiquement. Cela peut entraîner des incohérences, des retards de traitement et une surcharge de travail pour le personnel administratif. Enfin, le manque de visibilité pour le doctorant limite sa capacité à suivre l'évolution de son dossier en temps réel.

3.3 Solution proposée

Pour remédier à ces difficultés, nous proposons de développer Fiducia, une application web dédiée au suivi du parcours doctoral. Cette solution permettra de centraliser l'ensemble des données et documents relatifs à chaque doctorant, tout en automatisant les principales étapes du processus administratif. Le système offrira des fonctionnalités de gestion des comptes, de dépôt des dossiers, de suivi des validations et de contrôle des conditions de soutenance.

Grâce à cette plateforme, chaque utilisateur disposera d'un espace adapté à son rôle. Le doctorant pourra soumettre ses demandes et suivre leur traitement, le directeur de thèse pourra consulter et valider les dossiers nécessaires, tandis que l'administration pourra superviser l'ensemble du processus. Fiducia apportera ainsi une réponse structurée, moderne et efficace aux limites du système existant.

4. Choix du modèle de développement

Le modèle de développement retenu pour ce projet est une approche incrémentale. Ce choix consiste à concevoir et développer l'application par étapes successives, en intégrant

progressivement les différents modules fonctionnels. Chaque incrément est analysé, conçu, développé et testé avant de passer au suivant, ce qui permet de mieux maîtriser la complexité du projet.

Cette approche présente plusieurs avantages. Elle facilite l'organisation du travail, réduit les risques liés au développement global et permet de valider rapidement certaines fonctionnalités. Elle offre également une meilleure flexibilité pour intégrer d'éventuelles améliorations au fur et à mesure de l'avancement du projet. Dans le cadre de Fiducia, ce modèle est particulièrement adapté, car les modules de gestion des comptes, d'inscription et de soutenance peuvent être développés de manière progressive et cohérente.

5. Planning prévisionnel

Phase	Étape du Projet	Description / Objectifs
Cadrage	1. Analyse du cahier des charges	Définition du périmètre et validation des besoins utilisateurs.
Audit	2. Étude de l'existant	Analyse des processus actuels et diagnostic des points de blocage.
Conception	3. Conception & Modélisation	Architecture de la solution et création du schéma de données.
Réalisation	4. Développement	Codage des fonctionnalités clés et modules principaux.
Sécurisation	5. Intégration & Sécurité	Mise en place des règles métier et des protocoles de protection.
Validation	6. Tests & Recette	Vérification de la conformité (bugs, intégration, fonctions).
Clôture	7. Finalisation	Rédaction de la documentation et soutenance finale.

Tableau 1 : Planning prévisionnel

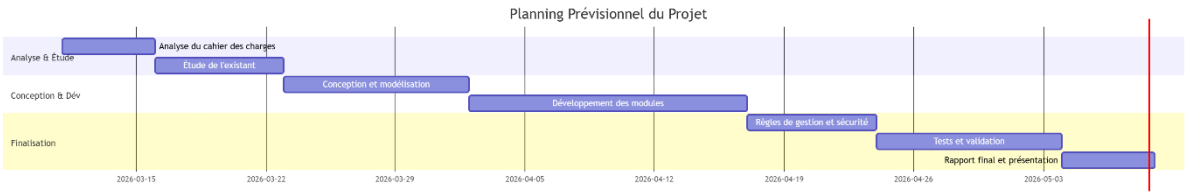


Figure 1 : Planning prévisionnel

6. Conclusion

Ce chapitre a permis de définir le cadre général du projet Fiducia et de mettre en évidence la nécessité d'une solution numérique centralisée pour le suivi du doctorat. L'étude de l'existant

a montré les limites des procédures manuelles, notamment en matière de suivi, de validation et de gestion documentaire. La solution proposée répond à ces insuffisances en offrant une plateforme web moderne, structurée et adaptée aux besoins des différents utilisateurs. Le choix d'un modèle de développement incrémental garantit une réalisation progressive et maîtrisée du système. Les prochains chapitres présenteront les besoins détaillés, la conception de la solution, puis sa réalisation et sa validation.

SPÉCIFICATION DES BESOINS

02

1. Introduction

Ce chapitre est consacré à la définition précise des besoins fonctionnels et non fonctionnels du portail **Fiducia**, dédié au suivi du parcours doctoral. La phase de spécification constitue une étape fondamentale, car elle permet de traduire les attentes des utilisateurs en exigences claires, structurées et exploitables lors de la conception et du développement du système. Dans le cadre de ce projet, les besoins exprimés concernent principalement la dématérialisation des démarches administratives liées à l'inscription, à la réinscription et à la soutenance, ainsi que l'amélioration du suivi des dossiers par les différents acteurs impliqués.

Les besoins fonctionnels décrivent les services que doit offrir l'application, notamment la gestion des comptes, l'authentification, la soumission des dossiers, le téléversement des pièces justificatives, le suivi de l'état d'avancement des demandes et la gestion du processus de soutenance. Les besoins non fonctionnels, quant à eux, portent sur la sécurité, la performance, l'ergonomie, la maintenabilité et la fiabilité de la plateforme. Ce chapitre présente également les acteurs du système ainsi que les cas d'utilisation permettant de modéliser les interactions entre les utilisateurs et l'application.

2. Spécification des besoins fonctionnels

Les besoins fonctionnels ont été définis en tenant compte des exigences exprimées dans le cahier des charges et des responsabilités de chaque profil utilisateur. Le système doit accompagner le doctorant tout au long de son parcours, depuis son inscription initiale jusqu'à la soutenance, tout en assurant le suivi et la validation des dossiers par le directeur de thèse et l'administration. L'objectif est de disposer d'une plateforme centralisée capable de simplifier les procédures, d'automatiser certaines vérifications et de garantir une meilleure traçabilité des actions effectuées.

2.1 Besoin fonctionnel 1 : Gestion des comptes et authentification

L'application doit permettre la création de compte pour les nouveaux candidats doctorants. Elle doit aussi offrir un mécanisme d'authentification sécurisé permettant à chaque utilisateur d'accéder à son espace personnel selon son rôle. Le système doit gérer plusieurs profils d'utilisateurs, notamment le doctorant, le directeur de thèse et le personnel administratif, avec des permissions adaptées à chaque type d'acteur.

En plus de la connexion et de la déconnexion sécurisées, l'administrateur doit pouvoir gérer les rôles et les autorisations. Cette gestion est essentielle pour garantir que chaque utilisateur

n'accède qu'aux fonctionnalités correspondant à ses responsabilités. Ce module constitue donc la base de fonctionnement de l'ensemble de la plateforme.

2.2 Besoin fonctionnel 2 : Gestion du processus d'inscription et de réinscription

L'application doit permettre la gestion complète des campagnes d'inscription et de réinscription. L'administration doit pouvoir définir les périodes d'ouverture et de fermeture des campagnes, afin d'encadrer les dépôts des dossiers. Le doctorant doit disposer d'un formulaire dynamique lui permettant de renseigner les informations nécessaires, telles que ses données personnelles, le sujet de thèse, le directeur de thèse ou encore les éventuelles collaborations.

Le système doit également permettre le téléversement des pièces justificatives requises, comme les diplômes, le CV ou la lettre de motivation, avec contrôle des formats autorisés. Une fois le dossier soumis, un circuit de validation en ligne doit être déclenché, comprenant l'avis du directeur de thèse puis la vérification et la validation administrative. Le doctorant doit enfin pouvoir consulter en temps réel l'état d'avancement de son dossier à partir d'un tableau de bord dédié.

2.3 Besoin fonctionnel 3 : Gestion du processus de soutenance

L'application doit prendre en charge les demandes de soutenance initiées par les doctorants. Elle doit proposer un formulaire dédié à cette étape finale du parcours doctoral, accompagné d'une gestion documentaire adaptée. Le doctorant doit notamment pouvoir déposer son manuscrit ainsi que les autres pièces exigées dans le cadre de la soutenance.

Le système doit intégrer une check-list de vérification des prérequis à la soutenance. Cette vérification porte sur le nombre de publications scientifiques, les crédits ou heures de formation doctorale obtenus, ainsi que la complétude des documents obligatoires. Le processus doit aussi permettre au directeur de thèse de donner son avis et de proposer les membres du jury, tandis que l'administration assure l'autorisation finale et la planification de la date, de l'heure et du lieu de la soutenance.

2.4 Besoin fonctionnel 4 : Suivi du dossier et visibilité sur l'avancement

L'application doit offrir au doctorant un espace de suivi clair et accessible lui permettant de connaître à tout moment l'état de traitement de ses demandes. Ce tableau de bord doit présenter les différentes étapes franchies, les validations obtenues, les pièces manquantes éventuelles ainsi que les actions encore attendues. Une telle fonctionnalité améliore la transparence du processus et réduit les besoins de relance manuelle.

De leur côté, le directeur de thèse et le personnel administratif doivent aussi disposer d'une visibilité suffisante sur les dossiers à traiter. Le système doit donc permettre un suivi structuré des demandes en attente, validées ou rejetées, afin de faciliter la gestion du processus global.

3. Spécification des besoins non fonctionnels

Les besoins non fonctionnels sont essentiels pour garantir la qualité globale du système. Dans le cas de Fiducia, la sécurité occupe une place centrale, puisque l'application manipule des données personnelles, académiques et administratives sensibles. Le système doit donc assurer une authentification sécurisée, une gestion rigoureuse des accès et une protection des documents téléversés.

La plateforme doit également être performante et réactive afin de permettre une utilisation fluide par plusieurs utilisateurs. L'ergonomie de l'interface doit faciliter la prise en main du système par des profils variés, y compris des utilisateurs non techniques. Enfin, l'application doit être maintenable et évolutive, ce qui justifie le recours à des technologies structurées comme Spring Boot, Spring Data JPA, Spring Security, Thymeleaf et une base de données relationnelle telle que PostgreSQL.

4. Présentation des cas d'utilisation

Les cas d'utilisation permettent de représenter de manière claire les interactions entre les différents acteurs et le système. Dans le projet Fiducia, ils servent à modéliser les grandes fonctionnalités du portail et à décrire les opérations principales réalisées par les doctorants, les directeurs de thèse et le personnel administratif. Cette modélisation facilite la compréhension des besoins métier et constitue un support important pour la conception du système.

Notre diagramme montre une organisation cohérente du portail autour de trois grands modules : l'authentification, l'inscription et la soutenance. Il met également en évidence les relations entre les acteurs et les fonctionnalités, ainsi que certaines dépendances importantes entre cas d'utilisation, notamment l'inclusion du téléversement des pièces justificatives dans la soumission du dossier, et l'inclusion de la vérification des prérequis dans la demande de soutenance.

4.1 Présentation des acteurs

Acteur	Rôle
Candidat / Doctorant	Soumet les dossiers d'inscription, de réinscription et de soutenance, dépose les pièces justificatives et suit l'état d'avancement de ses demandes.

Directeur de thèse	Consulte le statut du doctorant, donne son avis sur certaines demandes, signe les documents nécessaires et propose les membres du jury.
Personnel administratif	Vérifie la conformité des dossiers, valide administrativement les demandes, gère les campagnes d'inscription et planifie la soutenance.

Tableau 2 : Acteurs

4.2 Description des cas d'utilisation

Le **doctorant** interagit avec le système pour créer un compte, s'authentifier, soumettre son dossier d'inscription ou de réinscription, téléverser les pièces justificatives, suivre l'état de son dossier, soumettre une demande de soutenance et déposer son manuscrit. Ces cas d'utilisation constituent le cœur de son interaction avec la plateforme et traduisent les principales démarches administratives de son parcours doctoral.

Le **directeur de thèse** utilise le système pour s'authentifier, donner son avis sur les demandes d'inscription ou de soutenance et proposer les membres du jury. Le **personnel administratif**, quant à lui, s'authentifie, gère les campagnes d'inscription, valide administrativement les dossiers, vérifie les prérequis à la soutenance et planifie la soutenance. L'ensemble de ces interactions montre que Fiducia est conçu comme une plateforme collaborative où chaque acteur intervient selon un rôle bien défini dans le cycle de vie du doctorant.

4.3 Diagramme global des cas d'utilisation

Le diagramme de cas d'utilisation global du projet Fiducia illustre l'ensemble des interactions entre les trois acteurs principaux et les modules fonctionnels du portail. Il met en évidence la structuration du système autour du module d'authentification, du module d'inscription et du module de soutenance, tout en précisant les fonctionnalités accessibles à chaque profil. Cette représentation constitue un support visuel important pour comprendre l'organisation fonctionnelle globale de l'application.

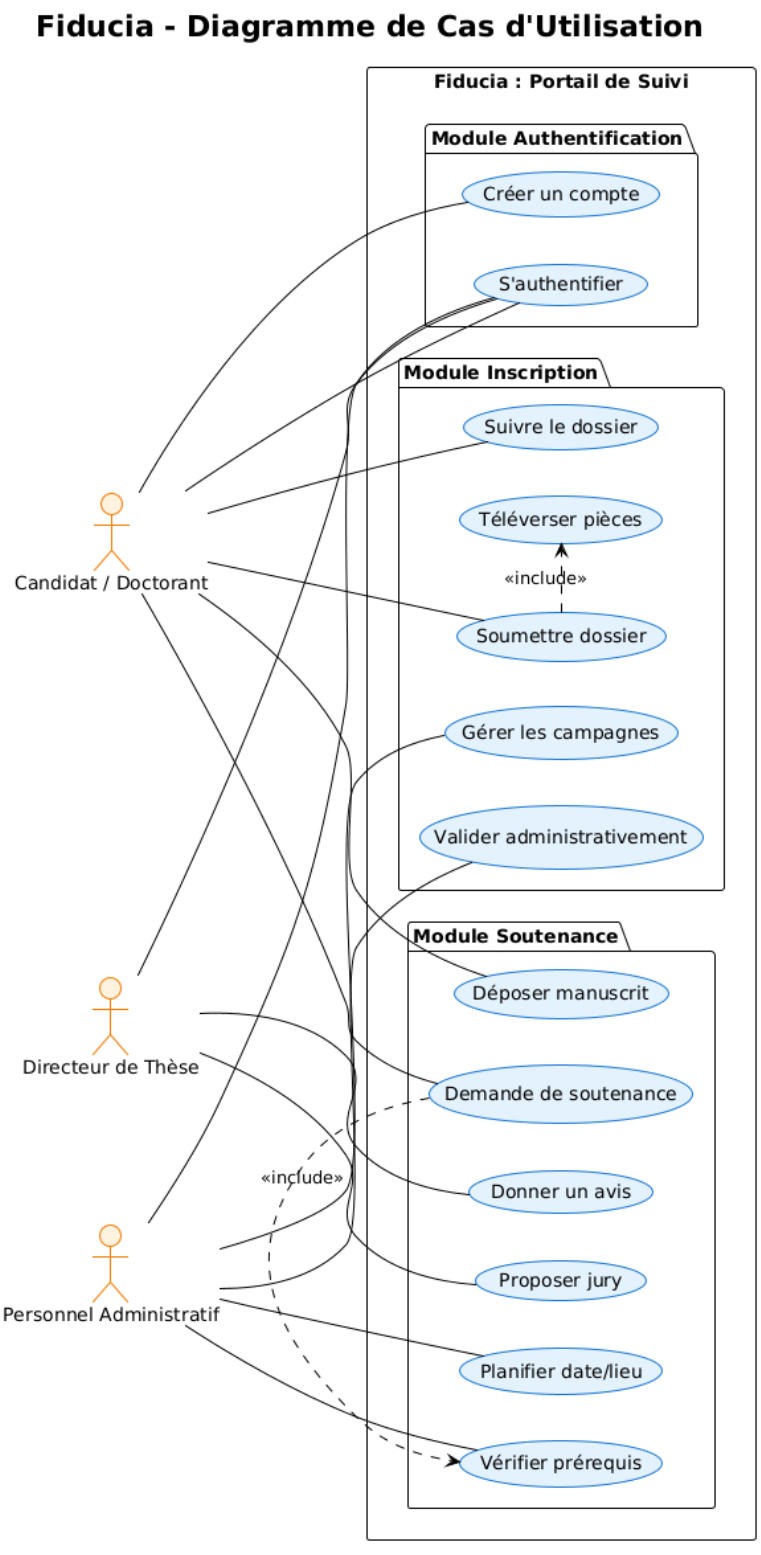


Figure 2 : Diagramme de cas d'utilisation

CONCEPTION DU SYSTÈME

03

1. Introduction

La phase de conception, présentée dans ce chapitre, constitue le lien entre la spécification des besoins et la réalisation technique du portail **Fiducia**. Elle a pour objectif de structurer le système de manière cohérente afin de garantir sa modularité, sa maintenabilité et son évolutivité. Dans le cadre de ce projet, la conception s'appuie sur des modélisations UML permettant de représenter à la fois le comportement dynamique du système et son organisation statique. Elle précise également l'architecture logicielle adoptée, fondée sur le modèle MVC, ainsi que l'environnement matériel et technique nécessaire au déploiement de l'application. Cette étape est essentielle pour anticiper les contraintes techniques, assurer la sécurité des échanges et préparer une implémentation conforme aux besoins définis dans les chapitres précédents.

2. Modélisation dynamique

La modélisation dynamique permet de représenter le comportement du système au cours de son exécution. Elle met en évidence les interactions entre les différents acteurs et composants logiciels, ainsi que l'enchaînement des traitements réalisés par l'application. Dans le cadre de Fiducia, les diagrammes dynamiques servent à illustrer les principales étapes du cycle de vie d'un dossier doctoral, depuis sa soumission jusqu'à sa validation finale. Ils permettent aussi de visualiser les flux d'information entre le doctorant, le directeur de thèse, l'administration, le contrôleur applicatif et la base de données. Cette représentation facilite la compréhension des processus métier et aide à valider la logique fonctionnelle du système avant son développement.

2.1. Diagramme de séquence

Le diagramme de séquence met en évidence l'enchaînement des interactions entre les différents acteurs et composants du système pour un scénario précis. Dans Fiducia, il illustre le processus de soumission d'un dossier d'inscription, l'avis du directeur de thèse, puis la validation administrative finale. Ce diagramme permet de comprendre comment une requête initiée par le doctorant est progressivement traitée par l'application jusqu'à l'enregistrement définitif du dossier dans la base de données.

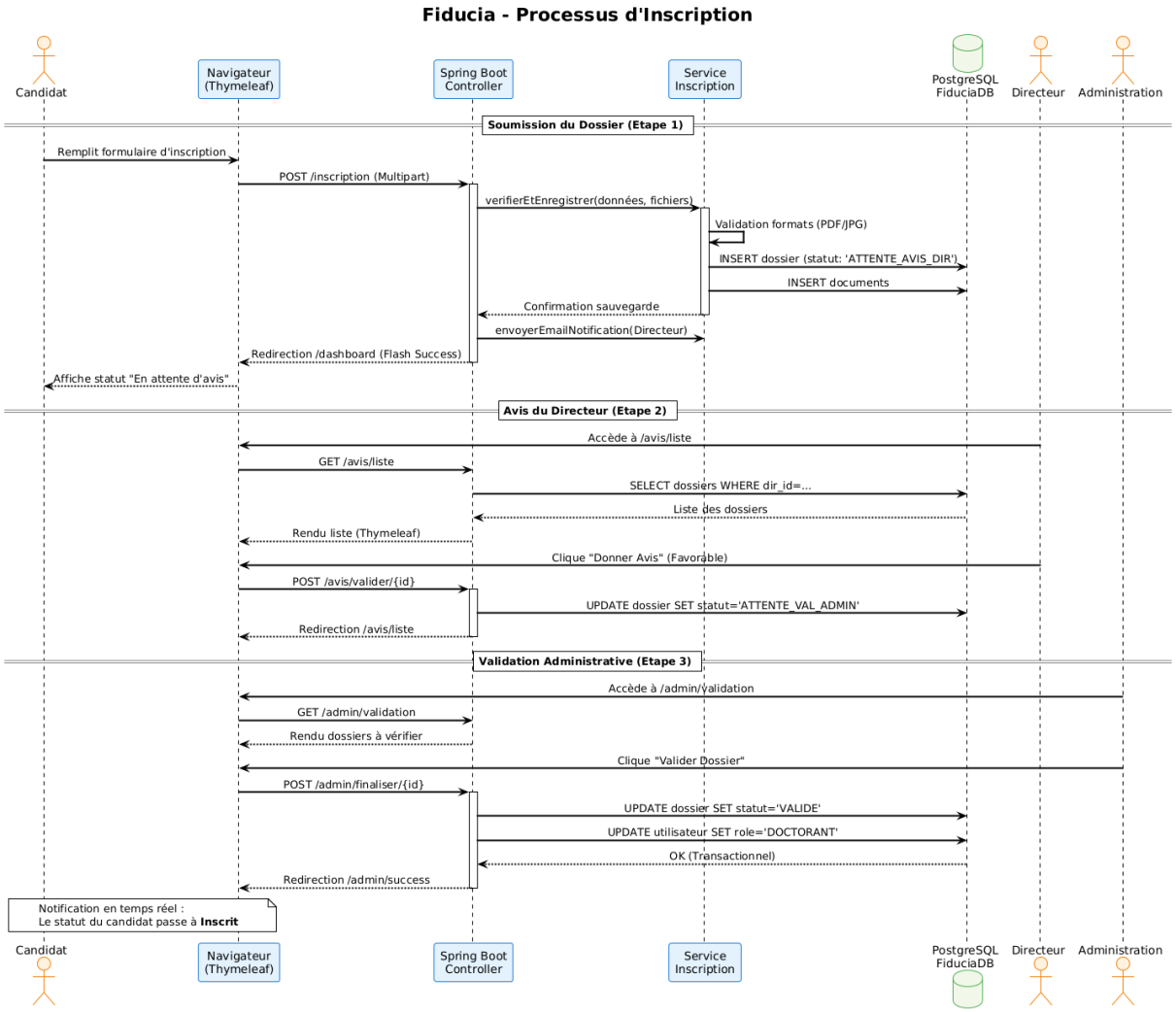


Figure 3 : Diagramme de séquence

Dans le premier temps, le candidat remplit le formulaire d'inscription à travers son navigateur. La requête est envoyée au contrôleur d'inscription sous la forme d'une requête HTTP de type POST contenant les données du formulaire et les fichiers joints. Le contrôleur transmet ensuite ces informations au service métier chargé de la vérification et de l'enregistrement du dossier. À ce niveau, le système contrôle la validité des formats des pièces justificatives, notamment les fichiers PDF et JPG, avant d'insérer les informations dans la base PostgreSQL. Une première ligne est ajoutée dans la table des dossiers avec un statut indiquant que le dossier est en attente d'avis du directeur, tandis que les documents associés sont enregistrés dans la table correspondante. Une notification est ensuite préparée pour informer le directeur de thèse, puis l'utilisateur est redirigé vers son tableau de bord avec un message de succès. À ce stade, le doctorant voit son dossier apparaître comme "en attente d'avis".

Dans un second temps, le directeur de thèse accède à la liste des dossiers qui lui sont attribués. L'application interroge la base de données afin de récupérer les demandes en attente, puis les

affiche dans une vue Thymeleaf. Le directeur peut alors consulter un dossier, examiner son contenu et donner un avis favorable. Lorsque cette décision est enregistrée, le statut du dossier est mis à jour afin d'indiquer qu'il attend désormais la validation administrative. Cette étape assure une séparation claire entre l'avis académique du directeur et le contrôle administratif effectué par le personnel habilité.

Enfin, dans la troisième étape, l'administration consulte les dossiers transmis pour validation finale. Après vérification, elle peut finaliser le traitement du dossier. Le système met alors à jour son statut en le passant à "validé", et le rôle de l'utilisateur peut également être modifié afin de refléter son intégration effective dans la base des doctorants. Une notification finale est renvoyée au candidat pour l'informer que son inscription est désormais confirmée. Ce fonctionnement garantit une traçabilité complète du processus et une gestion fiable des validations successives.

2.2 Diagramme d'activité

Le diagramme d'activité permet de représenter de manière visuelle le déroulement global des processus métier du système. Il met en évidence les différentes étapes, les décisions prises au cours du traitement, les enchaînements possibles ainsi que les alternatives selon les conditions rencontrées. Dans le cadre du projet Fiducia, ce diagramme modélise le cycle de vie complet du doctorant, depuis la création de compte et la soumission du dossier d'inscription jusqu'à la soutenance de thèse. Il constitue ainsi un support important pour comprendre la logique métier du portail et pour valider les workflows avec les besoins exprimés dans le cahier des charges.



Figure 4 : Diagramme d'activité

Le processus débute par la création du compte par le candidat, suivie du remplissage du formulaire d'inscription et du téléversement des pièces justificatives requises. Une première phase de validation est ensuite engagée. Le directeur de thèse intervient pour donner son avis sur le dossier soumis. Si cet avis est défavorable, le processus s'arrête ou fait l'objet d'une demande de correction. En revanche, si l'avis est favorable, le dossier est transmis à l'administration pour vérification de sa conformité. Cette dernière peut soit demander des compléments, soit procéder à la validation finale du dossier, ce qui permet l'attribution du rôle de doctorant à l'utilisateur.

Le diagramme intègre ensuite la dimension temporelle du parcours doctoral. Après validation de l'inscription, le doctorant poursuit son cycle de recherche en réalisant ses formations doctorales, en accumulant les crédits exigés et en produisant les publications scientifiques ainsi que les communications requises. Lorsque les conditions de soutenance ne sont pas encore remplies, le doctorant doit procéder à une réinscription annuelle. Le système tient compte ici des règles de gestion définies dans le cahier des charges, notamment la limitation à trois années sans dérogation et la durée maximale de six ans du doctorat. En cas de dépassement, une demande de dérogation peut être adressée au PED. Si cette dérogation n'est pas accordée, le parcours prend fin.

Lorsque le doctorant satisfait aux conditions de soutenance, il peut soumettre sa demande. Le système vérifie alors les prérequis, en particulier les 200 heures de formation, les publications scientifiques et les autres éléments exigés. Si ces conditions sont validées, le directeur de thèse propose les membres du jury, puis l'administration organise l'envoi du manuscrit aux rapporteurs. En cas de rapports favorables, l'administration délivre l'autorisation finale et fixe la date, l'heure et le lieu de la soutenance. Dans le cas contraire, une demande de révisions est émise. Ce diagramme d'activité reflète donc fidèlement la logique séquentielle et décisionnelle du parcours doctoral prise en charge par Fiducia.

3. Modélisation statique

La modélisation statique a pour objectif de représenter la structure interne du système à travers les entités métiers, leurs attributs, leurs opérations et les relations qui les lient. Elle constitue une base essentielle pour la conception de la base de données et l'organisation du code source. Dans le projet Fiducia, cette modélisation permet de formaliser les objets principaux du domaine doctoral, comme les utilisateurs, les dossiers d'inscription, les pièces justificatives, les demandes de soutenance, le jury ou encore le planning de soutenance.

Cette représentation statique favorise une meilleure compréhension du système et permet d'assurer la cohérence entre la conception conceptuelle et l'implémentation technique. Elle facilite également la définition des responsabilités de chaque classe et prépare le passage vers une architecture logicielle organisée, où les classes métier, les services et les composants de persistance interagissent de manière structurée.

3.1 Diagramme de classes

Le diagramme de classes présente les principales entités du système, leurs attributs, leurs méthodes ainsi que les relations d'association, de composition et d'héritage qui existent entre elles. Dans Fiducia, ce diagramme est structuré autour de trois grands modules : l'authentification et la gestion des rôles, le processus d'inscription, et le processus de soutenance. Cette organisation reflète directement les modules fonctionnels définis dans le cahier des charges.

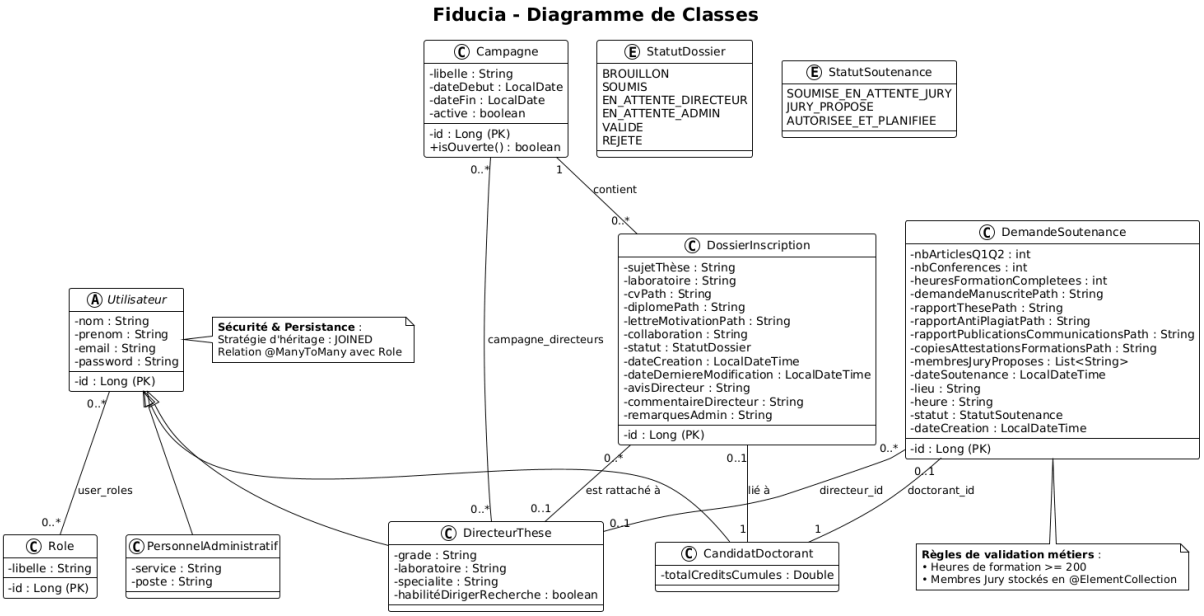


Figure 5 : Diagramme de classes

Le premier ensemble de classes concerne l'**authentification et la gestion des rôles**, conçu pour refléter fidèlement l'architecture de persistance sous-jacente. La classe abstraite Utilisateur constitue la base commune des différents profils du système et encapsule les attributs généraux essentiels tels que le nom, le prénom, l'adresse électronique et le mot de passe chiffré. Afin d'optimiser le modèle relationnel tout en préservant le polymorphisme, cette classe utilise une stratégie d'héritage de type JOINED. Elle se spécialise en trois sous-classes distinctes : PersonnelAdministratif (caractérisé par son service et son poste), CandidatDoctorant (qui suit l'état de son parcours via le total des crédits cumulés) et DirecteurThese (définissant son grade, son laboratoire, sa spécialité et son habilitation à diriger des recherches - HDR). Chaque sous-classe hérite des propriétés d'authentification communes tout en encapsulant les données propres à ses prérogatives métiers. Pour assurer une gestion souple et sécurisée des

habilitations, la classe Utilisateur maintient une relation de type réciproque (Many-to-Many) avec la classe Role, matérialisée par la table de jointure user_roles. Cette modélisation permet d'affecter dynamiquement des permissions spécifiques aux acteurs en parfaite conformité avec les exigences de sécurité du cahier des charges.

Le deuxième module est dédié au **processus d'inscription et de réinscription académique**. La classe Campagne modélise les fenêtres temporelles introduites par l'administration pour encadrer le dépôt des candidatures. Elle intègre des attributs de gestion des dates (dateDebut, dateFin), un indicateur d'activation et une méthode métier isOuverte() permettant de valider dynamiquement si la période courante autorise les flux de soumission. Chaque campagne est une structure d'accueil qui contient plusieurs dossiers, matérialisée par une relation d'un à plusieurs avec la classe DossierInscription. Cette dernière est le pivot central du module. Elle centralise les données académiques (sujet de thèse, laboratoire, collaboration éventuelle), le suivi des pièces jointes sous forme de chemins de stockage (cvPath, diplomePath, lettreMotivationPath), ainsi que le workflow décisionnel à travers l'énumération StatutDossier (allant du statut BROUILLON jusqu'à VALIDE ou REJETE). Le cycle de vie du dossier est rigoureusement tracé par des variables de diagnostic temporel (dateCreation, dateDerniereModification). Sur le plan relationnel, le dossier est structurellement lié à un unique CandidatDoctorant et se trouve rattaché aux fins d'évaluation à un DirecteurThese, qui formalise son expertise via les attributs avisDirecteur et commentaireDirecteur avant la validation finale par les remarques de l'administration.

Le troisième et dernier module structure le **processus de soutenance de thèse**, encadrant la fin du cycle doctoral. La classe DemandeSoutenance fait office de livrable de contrôle et regroupe l'ensemble des critères d'éligibilité scientifique et académique requis. Elle suit précisément les indicateurs de performance du doctorant, notamment le nombre d'articles publiés dans les revues indexées Q1/Q2 (nbArticlesQ1Q2), le volume de participations aux conférences (nbConferences), ainsi que le volume d'heures de formation complétées, soumis à une règle métier stricte exigeant un quota supérieur ou égal à 200 heures. La recevabilité de la demande est appuyée par la numérisation des pièces obligatoires stockées de manière unitaire (demandeManuscritePath, rapportThesePath, rapportAntiPlagiatPath, etc.). La liste des membres du jury proposés par le directeur de thèse est directement encapsulée au sein de l'entité sous forme de collection d'éléments (@ElementCollection). Une fois la demande auditée à travers les états de l'énumération StatutSoutenance (SOUMISE_EN_ATTENTE_JURY, JURY_PROPOSE), l'administration planifie concrètement la session de soutenance en fixant

les attributs de temps et d'espace (dateSoutenance, lieu, heure) pour basculer vers l'état AUTORISEE_ET_PLANIFIEE. Cette entité est connectée de façon univoque au CandidatDoctorant qui initie la démarche et au DirecteurThese qui en supervise la qualité scientifique.

Dans son ensemble, ce diagramme de classes fournit une transcription rigoureuse et exhaustive des spécifications fonctionnelles du portail Fiducia. En collant au plus près des annotations et des comportements des entités JPA, il offre une cartographie technique précise qui servira de fondement direct pour la génération du schéma de base de données relationnelle et le développement de la couche de services de l'application.

3.2 Modèle relationnel

Le modèle relationnel décrit la structure de la base de données utilisée par l'application **Fiducia**. Conformément au cahier des charges, l'application repose sur une base de données relationnelle de type **PostgreSQL**, permettant de stocker les informations relatives aux utilisateurs, aux rôles, aux campagnes d'inscription et aux dossiers soumis par les doctorants.

Le schéma relationnel du système peut être présenté comme suit :

- **utilisateur** (**id**, nom, prenom, email, password)
- **role** (**id**, libelle)
- **user_roles** (**user_id**, **role_id**)
- **candidat_doctorant** (**id**, total_credits_cumules)
- **directeur_these** (**id**, grade, laboratoire, specialite, habilite_diriger_recherche)
- **personnel_administratif** (**id**, service, poste)
- **campagne** (**id**, libelle, date_debut, date_fin, active)
- **campagne_directeurs** (**campagne_id**, **directeur_id**)
- **dossier_inscription** (**id**, candidat_id, directeur_id, sujet_these, laboratoire, cv_path, diplome_path, lettre_motivation_path, collaboration, statut, date_creation, date_derniere_modification, avis_directeur, commentaire_directeur, remarques_admin, campagne_id)

Dans ce modèle, la table **utilisateur** représente l'entité de base commune à tous les profils du système. Les tables **candidat_doctorant**, **directeur_these** et **personnel_administratif** héritent de cette table par spécialisation. La table **role** permet de définir les autorisations, tandis que **user_roles** assure la liaison entre les utilisateurs et leurs rôles. La table **campagne** encadre les périodes d'inscription, et la table **dossier_inscription** centralise les informations relatives aux dossiers soumis par les candidats.

3.3 Dictionnaire de données

Le dictionnaire de données présente les différentes tables de la base ainsi que les champs qui les composent, leurs types et leur signification. Il permet de mieux comprendre l'organisation des données manipulées par le système et de garantir une implémentation cohérente de la base de données.

Table	Champ	Type	Description
Utilisateur	id	BIGINT	Identifiant unique de l'utilisateur (clé primaire)
	nom	VARCHAR(255)	Nom de l'utilisateur
	prenom	VARCHAR(255)	Prénom de l'utilisateur
	email	VARCHAR(255)	Adresse e-mail unique de l'utilisateur
	password	VARCHAR(255)	Mot de passe de l'utilisateur
Role	id	BIGINT	Identifiant unique du rôle (clé primaire)
	libelle	VARCHAR(255)	Libellé du rôle, par exemple ROLE_ADMIN, ROLE_DOCTORANT ou ROLE_DIRECTEUR
User_Roles	user_id	BIGINT	Identifiant de l'utilisateur (clé étrangère vers Utilisateur)
	role_id	BIGINT	Identifiant du rôle (clé étrangère vers Role)
Candidat_Doctorant	id	BIGINT	Identifiant du candidat doctorant (clé primaire et clé étrangère vers Utilisateur)
	total_credits_cumules	DOUBLE	Total des crédits ou heures de formation cumulés
Directeur_These	id	BIGINT	Identifiant du directeur (clé primaire et clé étrangère vers Utilisateur)
	grade	VARCHAR(255)	Grade universitaire du directeur de thèse

	laboratoire	VARCHAR(255)	Laboratoire de rattachement
	specialite	VARCHAR(255)	Domaine de spécialité
	habilite_diriger_recherche	BOOLEAN	Indique si le directeur est habilité à diriger des recherches
Personnel_Administratif	id	BIGINT	Identifiant du personnel administratif (clé primaire et clé étrangère vers Utilisateur)
	service	VARCHAR(255)	Service d'affectation
	poste	VARCHAR(255)	Poste administratif occupé
Campagne	id	BIGINT	Identifiant unique de la campagne (clé primaire)
	libelle	VARCHAR(255)	Intitulé de la campagne d'inscription
	date_debut	DATE	Date de début de la campagne
	date_fin	DATE	Date de fin de la campagne
	active	BOOLEAN	Indique si la campagne est active
Campagne_Directeurs	campagne_id	BIGINT	Identifiant de la campagne (clé étrangère vers Campagne)
	directeur_id	BIGINT	Identifiant du directeur associé (clé étrangère vers Directeur_These)
Dossier_Inscription	id	BIGINT	Identifiant unique du dossier (clé primaire)
	candidat_id	BIGINT	Identifiant du candidat (clé étrangère vers Candidat_Doctorant)
	directeur_id	BIGINT	Identifiant du directeur de thèse (clé étrangère vers Directeur_These)
	sujet_these	VARCHAR(255)	Sujet de thèse proposé
	laboratoire	VARCHAR(255)	Laboratoire concerné par la thèse
	cv_path	VARCHAR(255)	Chemin de stockage du CV
	diplome_path	VARCHAR(255)	Chemin de stockage du diplôme
	lettre_motivation_path	VARCHAR(255)	Chemin de stockage de la lettre de motivation
	collaboration	VARCHAR(255)	Information sur une éventuelle collaboration
	statut	VARCHAR(255)	Statut du dossier : BROUILLON, SOUMIS, EN_ATTENTE_DIRECTEUR, EN_ATTENTE_ADMIN, VALIDE, REJETE
	date_creation	TIMESTAMP	Date de création du dossier

	date_derniere_modification	TIMESTAMP	Date de la dernière modification
	avis_directeur	VARCHAR(255)	Avis donné par le directeur de thèse
	commentaire_directeur	TEXT	Commentaire associé à l'avis du directeur
	remarques_admin	TEXT	Remarques formulées par l'administration
	campagne_id	BIGINT	Identifiant de la campagne liée au dossier (clé étrangère vers Campagne)

Tableau 3 : Dictionnaire de données

4. Architecture de l'application

L'architecture de l'application a été conçue pour répondre aux exigences fonctionnelles du portail de suivi du doctorat, tout en garantissant une structure claire, maintenable et évolutive. Elle repose sur une séparation nette entre la partie logicielle, qui gère les traitements et les interactions avec l'utilisateur, et la partie matérielle, qui assure l'hébergement des services nécessaires au fonctionnement de la plateforme.

4.1 Architecture logicielle

L'application est développée en utilisant le framework **Spring Boot**, qui s'appuie sur une architecture **Modèle-Vue-Contrôleur (MVC)**. Dans cette architecture, le **Modèle** représente les données et la logique métier, la **Vue** correspond à l'interface utilisateur, et le **Contrôleur** assure le rôle d'intermédiaire entre les requêtes de l'utilisateur, les traitements applicatifs et les réponses générées par le système.

Cette organisation permet de séparer les responsabilités de manière claire et facilite la maintenance du code ainsi que son évolution. Le framework **Spring MVC** est utilisé pour la gestion des requêtes HTTP et l'organisation des contrôleurs, tandis que **Thymeleaf** assure le rendu côté serveur des pages web. La persistance des données est assurée par **Spring Data JPA**, qui permet d'interagir avec la base de données relationnelle de manière structurée et simplifiée.

La sécurité du portail est assurée par **Spring Security**, utilisé pour gérer l'authentification, l'autorisation et la protection des différentes fonctionnalités selon les profils des utilisateurs. Ainsi, le doctorant, le directeur de thèse et le personnel administratif disposent chacun d'un accès adapté à leurs droits. Enfin, **Maven** est utilisé pour la gestion du cycle de vie du projet et des dépendances, tandis que **Docker** permet de faciliter le déploiement et l'isolation de l'application dans un environnement conteneurisé.

4.2 Architecture matérielle

L'architecture matérielle du système repose sur une organisation en plusieurs nœuds interconnectés afin d'assurer une séparation claire entre le poste utilisateur, le serveur d'application et le serveur de base de données. Cette configuration permet de garantir la disponibilité, la sécurité et la bonne répartition des traitements au sein du portail de suivi du doctorat.

Le **poste utilisateur** représente l'environnement client à partir duquel l'application est consultée. L'utilisateur accède au système via un **navigateur web**, qui héberge l'interface graphique réalisée en **HTML, CSS et JavaScript**. Ce poste peut être une machine de bureau ou un ordinateur portable disposant d'un navigateur moderne compatible avec les standards du web.

Le **serveur d'application** est conteneurisé à l'aide de **Docker**, ce qui facilite le déploiement, l'isolation et la portabilité de l'application. À l'intérieur de ce conteneur, l'exécution repose sur un environnement **Java Runtime (JRE)** qui héberge l'artefact principal de l'application, à savoir **Application.jar**. Cet artefact correspond à l'application développée avec **Spring Boot** et comprend les composants nécessaires au traitement des requêtes, notamment **Spring MVC / Thymeleaf** pour la gestion de l'interface serveur, **Spring Security** pour l'authentification et **Spring Data JPA** pour l'accès aux données.

Le **serveur de base de données** est également déployé dans un conteneur Docker distinct. Il contient le système de gestion de base de données **PostgreSQL**, chargé de stocker l'ensemble des informations du portail, telles que les utilisateurs, les candidatures, les dossiers d'inscription, les demandes de soutenance et les données administratives associées. Cette séparation entre l'application et la base de données améliore la sécurité et facilite la maintenance du système.

Les échanges entre les différents composants s'effectuent à travers le réseau selon les protocoles standards. Le navigateur web communique avec l'application via **HTTP** sur le **port 8080**, tandis que l'application accède à la base de données via **JDBC/SQL** sur le **port 5432**. Cette architecture permet d'obtenir une solution modulaire, performante et adaptée à un environnement de production ou de test.

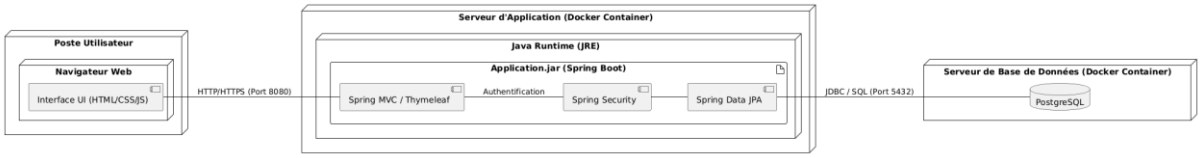


Figure 6 : Diagramme de déploiement

Le diagramme de déploiement UML représente l'architecture physique du système de suivi du doctorat. Il met en évidence les principaux nœuds matériels, à savoir le poste utilisateur, le serveur d'application et le serveur de base de données, ainsi que les échanges entre eux via le réseau. Le poste utilisateur accède à l'application à travers un navigateur web, tandis que le serveur héberge les composants applicatifs et la base de données.

Sur le serveur d'application, l'application Spring Boot est exécutée dans un environnement conteneurisé Docker. Elle comprend les composants **Spring MVC / Thymeleaf** pour la présentation, **Spring Security** pour la sécurité et **Spring Data JPA** pour l'accès aux données. Le serveur de base de données, également conteneurisé, héberge **PostgreSQL**, qui assure la persistance des informations du système. Ce schéma illustre clairement la répartition des responsabilités entre les différents éléments de l'architecture et montre comment les composants logiciels communiquent entre eux pour garantir le bon fonctionnement du portail.

5. Conclusion

Ce chapitre a défini les fondations techniques du portail de suivi du doctorat. L'adoption de l'architecture **Spring Boot (MVC)** garantit une séparation claire des responsabilités, tandis que l'utilisation de **Docker** pour l'application et la base de données **PostgreSQL** assure une infrastructure modulaire, sécurisée et facile à déployer.

Le diagramme de déploiement confirme cette organisation en optimisant les interactions entre les utilisateurs et les services. Cette structure robuste permet désormais d'aborder avec assurance la phase de réalisation et de mise en œuvre des modules fonctionnels.

RÉALISATION DU SYSTÈME

04

1. Introduction

Ce chapitre présente la phase de réalisation du portail de suivi du doctorat, en décrivant concrètement le développement de l'application ainsi que son environnement matériel et logiciel. Il met en évidence les choix techniques effectués pour répondre aux besoins fonctionnels définis précédemment, tout en assurant la fiabilité, la sécurité et la maintenabilité du système. Cette étape permet de transformer les spécifications et l'architecture en une solution opérationnelle, accessible aux différents acteurs du processus doctoral.

La réalisation du projet s'appuie sur un ensemble de technologies modernes permettant de structurer le développement, de faciliter le déploiement et d'améliorer l'expérience utilisateur. Le chapitre présente également les principales interfaces graphiques conçues pour les utilisateurs du portail, notamment celles destinées à l'authentification, à la gestion des dossiers d'inscription, au suivi des candidatures et à la demande de soutenance. L'objectif est de montrer comment les besoins métiers ont été traduits en fonctionnalités concrètes dans l'application finale.

2. Environnement de développement

2.1 Environnement matériel

Le développement et les tests du portail de suivi du doctorat ont été réalisés sur une machine personnelle de type **MSI Modern 15 H C13M**, équipée d'un processeur **13th Gen Intel Core i7-13620H**, de **16 Go de RAM** et fonctionnant sous **Windows 11 Famille 64 bits**. Cette configuration offre une capacité suffisante pour exécuter simultanément l'environnement de développement, le serveur d'application, les outils de test et les composants liés au déploiement local.

Bien que cette machine soit destinée à un usage personnel, elle constitue un environnement adapté au développement d'une application web en phase de conception et de test. La mémoire disponible permet de lancer les services nécessaires sans ralentissement majeur, tandis que le processeur assure une bonne réactivité lors de la compilation, du lancement de l'application et des vérifications fonctionnelles.

2.2 Environnement logiciel

L'environnement logiciel repose sur une pile technologique adaptée au développement d'une application web structurée autour de **Spring Boot**. Le projet utilise **Java** comme langage principal, avec le framework **Spring Boot** pour la logique applicative, **Spring MVC** pour la

gestion des requêtes web, **Spring Security** pour l'authentification et l'autorisation, et **Spring Data JPA** pour l'accès aux données. **Thymeleaf** est utilisé pour le rendu des vues côté serveur, ce qui permet de générer des interfaces dynamiques et cohérentes avec les besoins du système.

Le projet s'appuie également sur **Maven** pour la gestion des dépendances et l'automatisation des tâches de construction, ainsi que sur **Docker** pour le déploiement des composants dans des conteneurs isolés. La base de données relationnelle utilisée est **PostgreSQL**, ce qui assure une bonne robustesse pour la persistance des données du portail. Des outils comme **Git** et **Postman** facilitent respectivement la gestion du code source et les tests des services applicatifs.

Technologies utilisées

- **Java / Spring Boot** : langage et framework principaux pour la logique métier, les contrôleurs et la structure générale de l'application.
- **Spring MVC / Thymeleaf** : gestion des requêtes web et génération des pages côté serveur.
- **Spring Security** : protection de l'accès aux ressources et gestion des rôles utilisateurs.
- **Spring Data JPA** : accès simplifié aux données et gestion des entités métier.
- **Maven** : outil de build et de gestion des dépendances.
- **Docker** : conteneurisation de l'application et de la base de données.
- **PostgreSQL** : système de gestion de base de données relationnelle.
- **Git** : suivi des versions du code source.
- **Postman** : test des routes et validation des échanges HTTP.



Figure 10 : Spring boot



Figure 7 : Git



Figure 9 : Docker



Figure 8 : Java

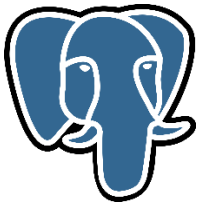


Figure 14 : Postgresql



Figure 11 : Postman



Figure 12 : Thymeleaf



Figure 13 : Maven



Figure 15 : Spring Security



Figure 16 : Spring data JPA

3. Principales interfaces graphiques

Les interfaces graphiques ont été conçues pour offrir une expérience utilisateur simple, lisible et fonctionnelle. L'écran d'accueil et les pages d'authentification constituent le point d'entrée de l'application, permettant à chaque utilisateur d'accéder à son espace selon son profil. L'interface de tableau de bord centralise les informations essentielles et donne une vue synthétique de l'état des dossiers en cours de traitement.

L'interface de gestion des inscriptions et réinscriptions permet au doctorant de soumettre ses informations, de joindre les pièces justificatives et de suivre l'évolution de son dossier. De son côté, l'interface dédiée à la soutenance regroupe les formulaires de demande, les documents à fournir et les étapes de validation administrative. L'ensemble a été développé avec un souci particulier d'ergonomie, de clarté visuelle et de cohérence avec les besoins des différents acteurs du portail.

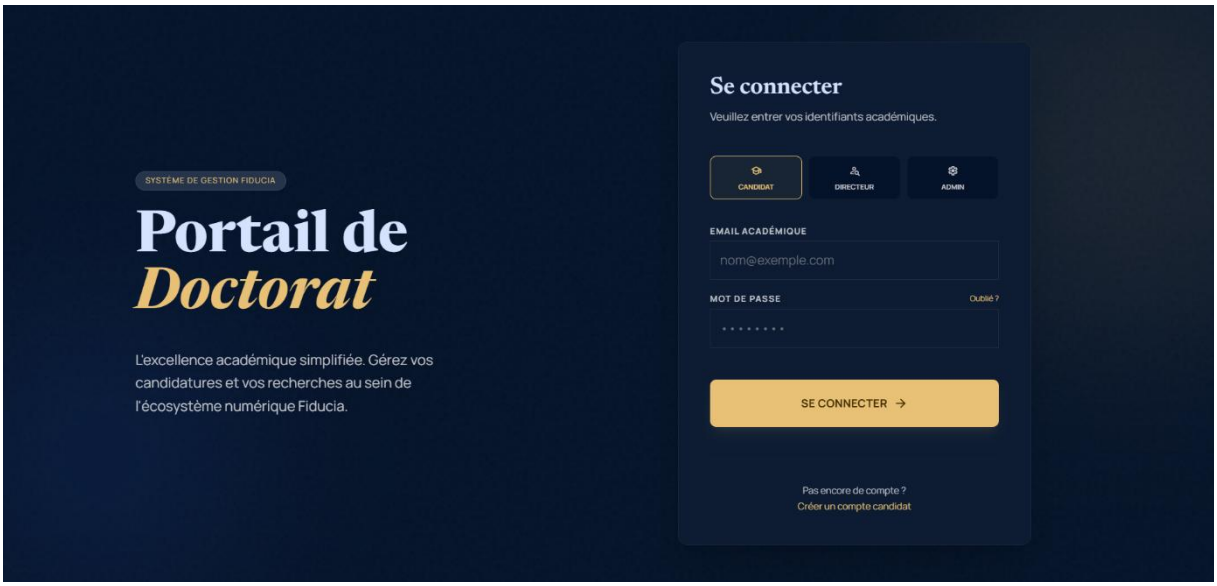


Figure 17 : Page login

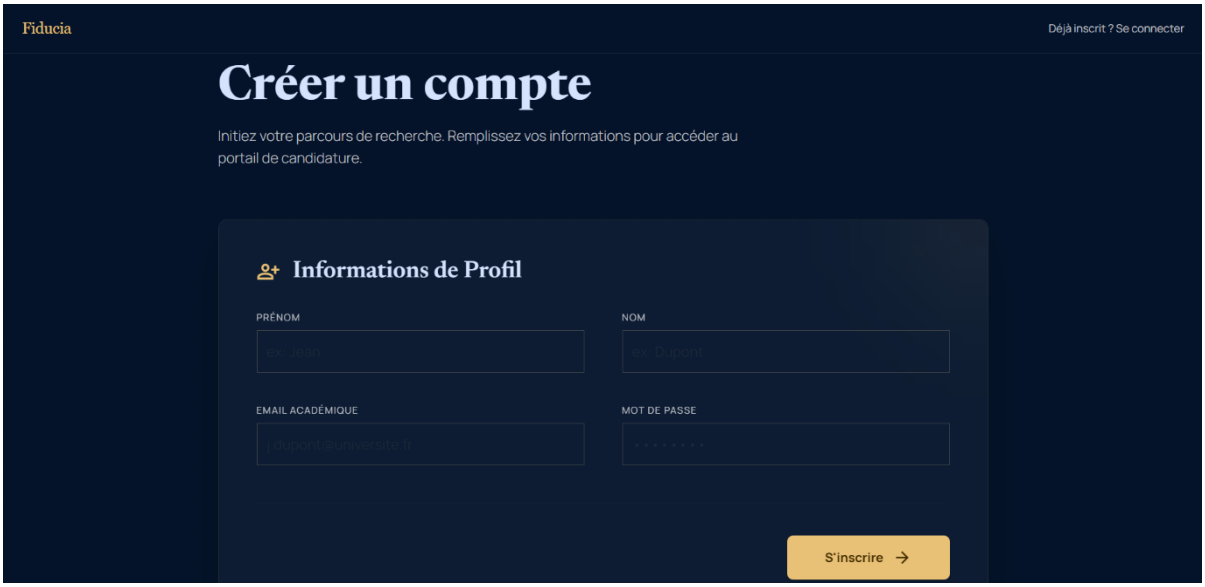


Figure 18 : Page Sign in

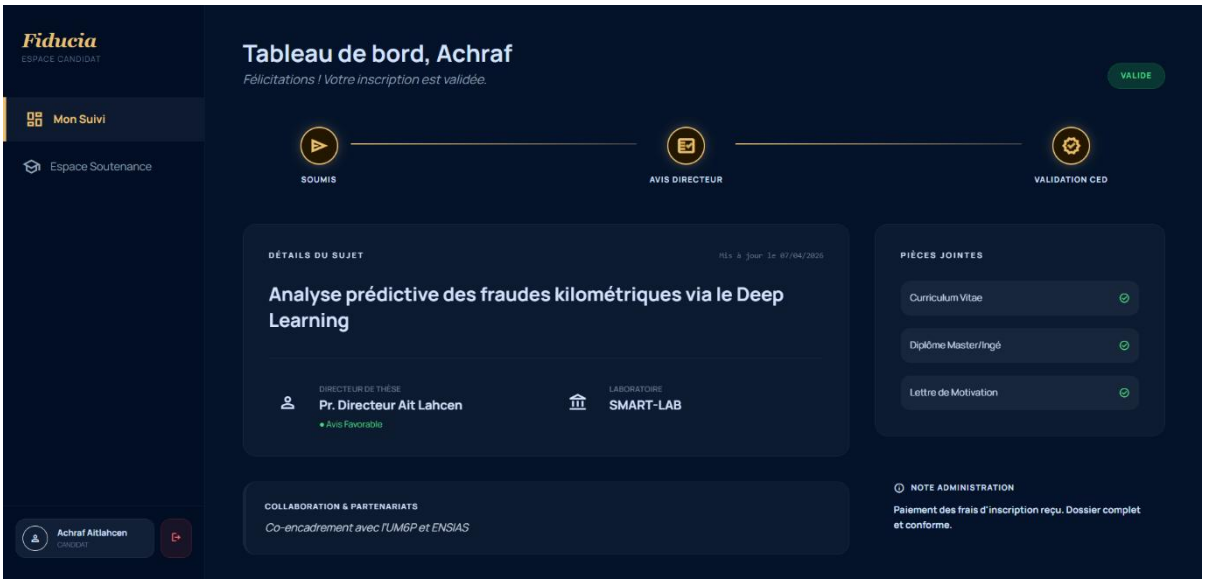


Figure 19 : Dashboard Doctorant

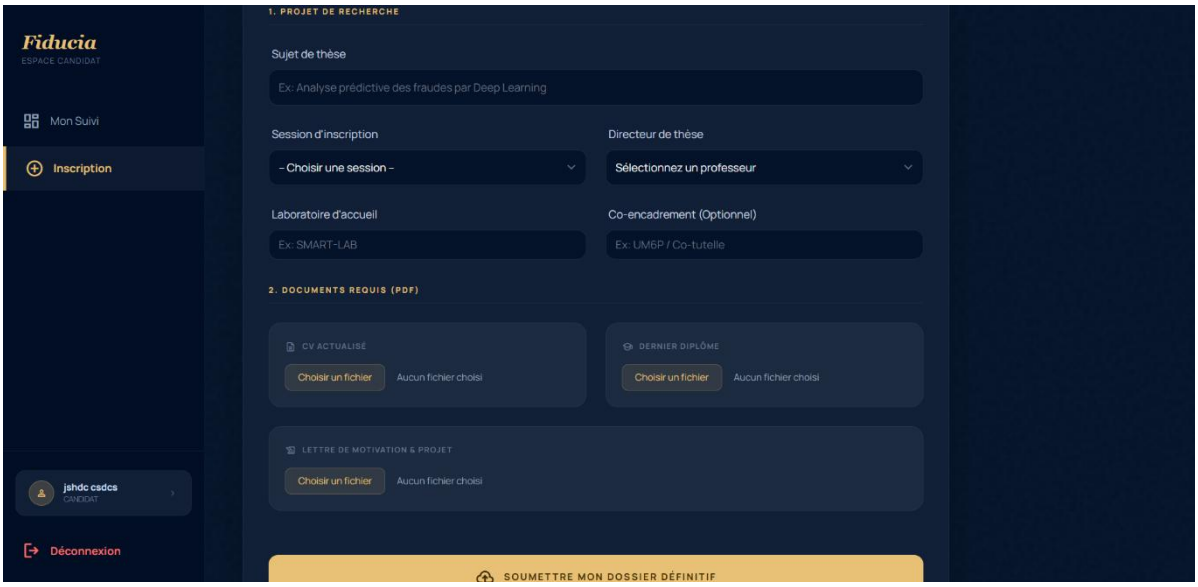


Figure 20 : Dossier d'inscription

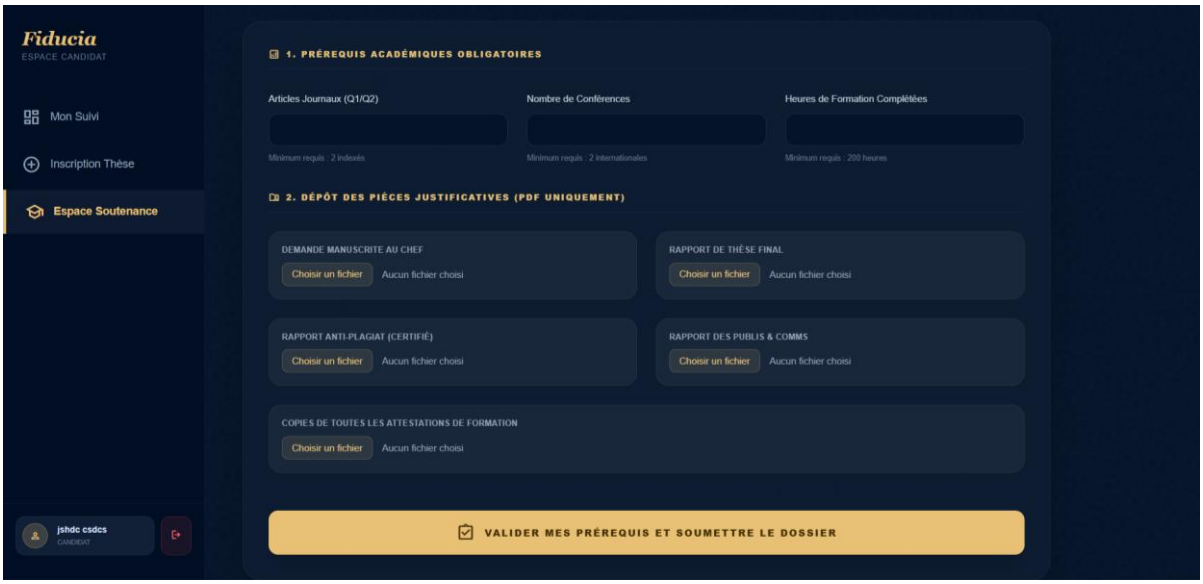


Figure 21 : Demande de soutenance

4. Conclusion

Ce chapitre a présenté la phase de réalisation du portail de suivi du doctorat en détaillant l'environnement de développement, les technologies utilisées et les principales interfaces graphiques mises en œuvre. L'environnement matériel, basé sur une machine personnelle performante, a permis de développer et de tester l'application dans de bonnes conditions. L'environnement logiciel, fondé sur **Spring Boot**, **Thymeleaf**, **Spring Security**, **Spring Data JPA**, **Docker** et **PostgreSQL**, a fourni une base solide pour la mise en place des fonctionnalités du système.

Les interfaces développées traduisent concrètement les besoins fonctionnels du projet en proposant des écrans adaptés à chaque étape du parcours doctoral. Elles constituent une première version opérationnelle du portail et ouvrent la voie à des améliorations futures, notamment en matière de test, d'optimisation et de déploiement en environnement de production.

TESTS ET VALIDATION

05

1. Introduction

Ce chapitre est consacré à la phase de tests et de validation du portail de suivi du doctorat. Après la conception architecturale et la réalisation des différentes fonctionnalités, il est indispensable de vérifier que l'application répond correctement aux besoins définis dans les chapitres précédents et qu'elle fonctionne de manière fiable dans des conditions d'utilisation réelles ou simulées. Les tests permettent de s'assurer que chaque fonctionnalité se comporte comme prévu, que les règles de gestion sont respectées et que les interactions entre les différents modules ne provoquent pas d'anomalies.

La validation constitue une étape essentielle dans le cycle de vie du logiciel, car elle permet d'évaluer la conformité du système par rapport au cahier des charges. Dans le cadre de ce projet, les tests ont porté principalement sur les modules d'authentification, de gestion des utilisateurs, de gestion des campagnes d'inscription, de soumission des dossiers par les candidats, de validation par les directeurs de thèse et de finalisation administrative. Cette étape a également permis de vérifier l'accès aux ressources selon les rôles attribués à chaque utilisateur, ainsi que le bon traitement des données transmises à l'application.

2. Objectifs des tests

Les tests réalisés avaient pour principal objectif de vérifier le bon fonctionnement global de l'application ainsi que la conformité des résultats obtenus avec les attentes fonctionnelles. Plus précisément, ils visaient à :

- Vérifier que les utilisateurs peuvent s'inscrire et se connecter correctement ;
- Contrôler que les rôles et permissions sont attribués et gérés de manière cohérente ;
- Valider le bon fonctionnement des opérations d'administration ;
- Tester la création, la consultation et la gestion des campagnes d'inscription ;
- S'assurer qu'un candidat peut soumettre un dossier complet avec ses pièces justificatives ;
- Vérifier que le directeur de thèse peut émettre un avis sur les dossiers ;
- Contrôler que l'administration peut finaliser le traitement d'un dossier ;
- Valider l'accès au profil utilisateur et la mise à jour des informations personnelles ;
- Confirmer que les réponses de l'API respectent les formats attendus.

Ces objectifs permettent de couvrir à la fois les fonctionnalités principales du système et les interactions critiques entre les différents profils d'utilisateurs.

3. Environnement de test

Les tests ont été effectués dans un environnement local de développement, à l'aide d'un serveur exécuté sur l'adresse http://localhost:8080. Les requêtes ont été envoyées sous forme de tests API, ce qui a permis de valider directement le comportement du backend sans passer par l'interface graphique.

L'environnement de test s'appuie sur :

- Un serveur Spring Boot en exécution locale ;
- Une base de données PostgreSQL configurée pour le projet ;
- Un outil de test des API permettant de simuler les requêtes HTTP (Postman);
- Des comptes de test créés pour chaque rôle utilisateur ;
- Des données de test représentatives du fonctionnement réel du portail.

Les formats utilisés varient selon les cas :

- **JSON** pour les opérations d'authentification, de création de comptes, de gestion des campagnes et de validation ;
- **multipart/form-data** pour la soumission du dossier candidat et le téléversement des documents.

Cet environnement a permis de réaliser des vérifications précises et reproductibles sur l'ensemble des fonctionnalités principales.

4. Méthodologie de test

La méthodologie adoptée repose sur une approche fonctionnelle, consistant à tester chaque service de manière isolée puis à vérifier son intégration avec les autres modules. Chaque test est défini par :

- Un objectif précis ;
- Un endpoint ciblé ;
- Des données d'entrée ;

- Un résultat attendu ;
- Un résultat observé ;
- Un statut de validation.

Cette méthode permet de détecter rapidement les erreurs éventuelles et de confirmer que les fonctionnalités respectent bien les règles métier définies dans le projet. Les cas de test ont été choisis de manière à couvrir les opérations essentielles du cycle de vie d'un dossier doctoral.

5. Cas de test fonctionnels

5.1. Test d'inscription d'un candidat

L'un des premiers tests réalisés concerne l'inscription d'un candidat sur la plateforme. L'objectif est de vérifier qu'un utilisateur peut créer un compte de type candidat en fournissant ses informations personnelles et un mot de passe valide.

Endpoint : POST http://localhost:8080/api/auth/register

Données envoyées :

```
{
  "nom": "Ez",
  "prenom": "Mehdi",
  "email": "candidat@emsi.ma",
  "password": "candidatpassword"
}
```

Résultat attendu : Le compte du candidat doit être créé avec succès et enregistré dans la base de données.

Résultat obtenu : L'inscription a été effectuée correctement, et le compte candidat a été ajouté au système.

Statut : Validé.

5.2. Test d'inscription d'un directeur de thèse

Ce test permet de vérifier la création d'un compte pour un directeur de thèse, avec les attributs spécifiques liés à ce profil.

Endpoint : POST http://localhost:8080/api/auth/register-directeur

Données envoyées :

```
{
  "nom": "Ait Lahcen",
  "prenom": "Directeur Test",
  "email": "directeur@emsi.ma",
  "password": "123",
  "grade": "PES",
  "laboratoire": "Labo MIAGE",
  "specialite": "Systèmes d'Information",
  "habilitéDirigerRecherche": true
}
```

Résultat attendu : Le directeur doit être enregistré avec ses informations académiques et professionnelles.

Résultat obtenu : Le compte directeur a été créé avec succès.

Statut : Validé.

5.3. Test d'inscription d'un administrateur

Ce test vérifie la création du compte administrateur chargé de la gestion globale de la plateforme.

Endpoint :

POST http://localhost:8080/api/auth/register-admin

Données envoyées :

```
{
  "nom": "Admin",
  "prenom": "Scolarité",
  "email": "admin@emsi.ma",
  "password": "123",
  "service": "Service Doctorat",
  "poste": "Chef de département"
```

```
}
```

Résultat attendu : Le compte administrateur doit être créé avec les informations administratives nécessaires.

Résultat obtenu : Le compte administrateur a été créé avec succès.

Statut : Validé.

5.4. Test de connexion

Une fois les comptes créés, un test de connexion a été effectué afin de vérifier le mécanisme d'authentification.

Endpoint : POST http://localhost:8080/api/auth/login

Données envoyées :

```
{
  "email": "admin@emsi.ma",
  "password": "123"
}
```

Résultat attendu : L'utilisateur doit être authentifié et obtenir l'accès à son espace selon son rôle.

Résultat obtenu : La connexion a été acceptée et l'authentification s'est déroulée correctement.

Statut : Validé.

5.5. Test de consultation des utilisateurs

Ce test permet de vérifier que l'administrateur peut consulter la liste des utilisateurs enregistrés sur la plateforme.

Endpoint: GET http://localhost:8080/api/admin/users

Résultat attendu : La liste des utilisateurs doit être affichée.

Résultat obtenu : Les utilisateurs ont été récupérés correctement par l'API.

Statut : Validé.

5.6. Test d'ajout d'un rôle à un utilisateur

L'administration doit pouvoir attribuer un rôle supplémentaire à un utilisateur donné.

Endpoint : POST http://localhost:8080/api/admin/users/1/add-role

Données envoyées :

```
{
  "roleName": "ROLE_DIRECTEUR"
}
```

Résultat attendu : Le rôle doit être ajouté au profil de l'utilisateur concerné.

Résultat obtenu : Le rôle a été attribué avec succès.

Statut : Validé.

5.7. Test de suppression d'un rôle

Ce test vérifie la possibilité de retirer un rôle à un utilisateur.

Endpoint : POST http://localhost:8080/api/admin/users/1/remove-role

Données envoyées :

```
{
 "roleName": "ROLE_DIRECTEUR"
}
```

Résultat attendu : Le rôle doit être supprimé du profil utilisateur.

Résultat obtenu : La suppression du rôle a été effectuée correctement.

Statut : Validé.

5.8. Test de création d'une campagne d'inscription

L'administration doit pouvoir ouvrir une nouvelle campagne d'inscription pour une période définie.

Endpoint : POST http://localhost:8080/api/admin/campagnes

Données envoyées :

```
{
  "libelle": "Inscription Doctorat 2026",
 "dateDebut": "2026-01-01",
  "dateFin": "2026-12-31",
```

```
  "directeursAutorises": [
    { "id": 1 }
  ],
  "active": true
}
```

Résultat attendu : La campagne doit être créée et enregistrée avec son statut actif.

Résultat obtenu : La campagne a été ajoutée avec succès.

Statut : Validé.

5.9. Test de consultation des campagnes

Ce test permet de vérifier que l'administration peut consulter l'ensemble des campagnes créées.

Endpoint: GET http://localhost:8080/api/admin/campagnes

Résultat attendu : La liste des campagnes doit être retournée.

Résultat obtenu : Les campagnes ont été récupérées correctement.

Statut : Validé.

5.10. Test de récupération de la campagne active

L'application doit permettre d'identifier rapidement la campagne en cours.

Endpoint: GET http://localhost:8080/api/admin/campagnes/active

Résultat attendu : La campagne active doit être affichée.

Résultat obtenu : La campagne active a été retournée correctement.

Statut : Validé.

5.11. Test de désactivation d'une campagne

Une campagne peut être désactivée une fois la période terminée.

Endpoint: PATCH http://localhost:8080/api/admin/campagnes/5/desactiver

Résultat attendu : La campagne doit passer à l'état inactif.

Résultat obtenu : La désactivation a été réalisée avec succès.

Statut : Validé.

5.12. Test de soumission d’un dossier candidat

Le candidat doit pouvoir soumettre un dossier complet contenant ses informations académiques et les pièces justificatives nécessaires.

Endpoint : POST http://localhost:8080/api/candidat/soumettre-dossier

Données envoyées :

- sujet : Optimisation des SI via l'IA générative
- directeurId : 2
- laboratoire : LABO-STIC
- collaboration : Partenariat avec UM6P
- cv : fichier PDF
- diplome : fichier JPG ou PDF
- lettre : fichier PDF

Résultat attendu : Le dossier doit être enregistré avec succès et associé au candidat.

Résultat obtenu : La soumission a été prise en compte correctement.

Statut : Validé.

5.13. Test de consultation du statut du candidat

Ce test vérifie que le candidat peut suivre l’état de traitement de son dossier.

Endpoint: GET http://localhost:8080/api/candidat/mon-statut

Résultat attendu : Le statut du dossier doit être affiché.

Résultat obtenu : Le statut a bien été retourné.

Statut : Validé.

5.14. Test de suivi détaillé du dossier

Le candidat doit également pouvoir consulter le suivi détaillé de son dossier.

Endpoint: GET http://localhost:8080/api/candidat/mon-suivi

Résultat attendu : Le suivi du dossier doit être affiché de manière claire.

Résultat obtenu : Les informations de suivi ont été récupérées correctement.

Statut : Validé.

5.15. Test d'avis du directeur

Le directeur de thèse doit pouvoir émettre un avis sur un dossier soumis.

Endpoint : POST http://localhost:8080/api/validation/directeur/3/avis

Données envoyées :

```
{
  "avis": "FAVORABLE",
  "commentaires": "Excellent sujet, très pertinent pour le
laboratoire SMART-LAB. Le candidat possède les prérequis nécessaires
en Deep Learning.",
  "valide": true
}
```

Résultat attendu : L'avis du directeur doit être enregistré dans le système.

Résultat obtenu : L'avis a été validé et enregistré correctement.

Statut : Validé.

5.16. Test de finalisation administrative

Après l'avis du directeur, l'administration doit pouvoir finaliser le dossier.

Endpoint : POST http://localhost:8080/api/validation/admin/5/finaliser

Données envoyées :

```
{
  "valide": true,
  "remarques": "Paiement des frais d'inscription reçu. Dossier
complet et conforme."
}
```

Résultat attendu : Le dossier doit être finalisé et validé administrativement.

Résultat obtenu : La finalisation a été réalisée avec succès.

Statut : Validé.

5.17. Test de consultation du profil utilisateur

Chaque utilisateur doit pouvoir consulter son propre profil.

Endpoint: GET http://localhost:8080/api/auth/me/profile

Résultat attendu : Les informations du profil doivent être retournées.

Résultat obtenu : Le profil a été affiché correctement.

Statut : Validé.

5.18. Test de mise à jour du profil

L'utilisateur doit pouvoir mettre à jour certaines informations de son profil.

Endpoint: PUT http://localhost:8080/api/auth/me/profile

Données envoyées :

```
{
  "totalCreditsCumules": 25000.00
}
```

Résultat attendu : Le profil doit être mis à jour dans la base de données.

Résultat obtenu : La mise à jour a été effectuée correctement.

Statut : Validé.

5.19. Test de consultation des dossiers du directeur

Le directeur doit pouvoir accéder aux dossiers qui lui sont affectés.

Endpoint: GET http://localhost:8080/api/validation/directeur/mes-dossiers

Résultat attendu : La liste des dossiers associés au directeur doit être affichée.

Résultat obtenu : Les dossiers ont été récupérés correctement.

Statut : Validé.

6. Synthèse des validations

Les tests réalisés ont permis de vérifier le bon fonctionnement des principales fonctionnalités du portail de suivi du doctorat. Les opérations d'inscription, de connexion et de gestion des rôles fonctionnent conformément aux attentes. La gestion des campagnes d'inscription a également été validée, tout comme la soumission des dossiers par les candidats et le suivi de leur état d'avancement. De plus, les mécanismes de validation par le directeur et par l'administration ont montré que le processus de traitement des dossiers est bien respecté.

L’ensemble des vérifications effectuées confirme que l’application répond de manière satisfaisante aux exigences fonctionnelles définies au préalable. Les résultats obtenus montrent que le système est capable de gérer les interactions entre les différents utilisateurs de façon cohérente, sécurisée et structurée.

7. Conclusion

Ce chapitre a présenté la phase de tests et de validation du portail de suivi du doctorat. Les tests fonctionnels réalisés sur les différents endpoints ont permis de confirmer le bon fonctionnement des modules essentiels du système, depuis l’authentification jusqu’à la validation finale des dossiers. Les résultats obtenus montrent que l’application respecte globalement les besoins exprimés dans le cahier des charges et qu’elle constitue une base fiable pour une utilisation réelle.

Cette phase de validation a également permis d’identifier les points sensibles du système, notamment la cohérence des données transmises, la gestion des rôles et la circulation des informations entre les différents acteurs. Ainsi, les tests réalisés constituent une étape importante avant la mise en production de la plateforme.

Conclusion générale

Le projet de portail de suivi du doctorat atteint pleinement ses objectifs en proposant une solution web structurée, cohérente et adaptée à la dématérialisation du parcours doctoral. Il démontre une bonne maîtrise des concepts essentiels du génie logiciel appliqués à une plateforme de gestion académique, notamment l'architecture multicouche, la séparation des responsabilités, la gestion des rôles et la persistance des données. L'ensemble du travail réalisé montre que les besoins fonctionnels identifiés au départ ont été traduits avec succès en une application opérationnelle, capable de centraliser les démarches d'inscription, de réinscription et de soutenance.

Le portail met en œuvre une organisation claire du système autour de plusieurs acteurs principaux, à savoir le candidat doctorant, le directeur de thèse et le personnel administratif. Chacun dispose d'un espace et de fonctionnalités adaptés à son rôle, ce qui permet d'assurer une circulation fluide de l'information ainsi qu'un suivi plus transparent des dossiers. La gestion des comptes, l'authentification, la soumission des dossiers, le téléversement des pièces justificatives, l'avis du directeur et la validation administrative constituent les principales composantes fonctionnelles du système. Cette structuration garantit une meilleure traçabilité des actions et une réduction des traitements manuels.

Sur le plan technique, le projet s'appuie sur une architecture moderne fondée sur **Spring Boot**, le modèle **MVC**, **Spring Security**, **Spring Data JPA**, **Thymeleaf**, **Docker** et **PostgreSQL**. Cette combinaison technologique assure une base solide pour le développement, la modularité du code et la maintenabilité de l'application. Le choix d'une séparation nette entre le poste utilisateur, le serveur d'application et le serveur de base de données renforce également la clarté de l'architecture et facilite le déploiement dans un environnement local ou conteneurisé.

La phase de tests et de validation a confirmé le bon fonctionnement des principaux services exposés par l'application. Les tests d'inscription, de connexion, de gestion des rôles, de création de campagnes, de soumission de dossiers et de validation ont montré que le système répond correctement aux scénarios attendus. Les résultats obtenus confirment que la plateforme respecte globalement les exigences définies dans le cahier des charges et qu'elle peut servir de base fiable pour une utilisation réelle.

Au-delà de l'aspect académique, ce projet constitue un socle technique réutilisable et évolutif pour des contextes similaires de gestion administrative et universitaire. Certaines améliorations

restent envisageables, notamment le renforcement de la sécurité, l'ajout de notifications automatiques, l'enrichissement du tableau de bord de suivi et l'optimisation de l'expérience utilisateur. Ces évolutions permettront d'améliorer encore la maturité de la solution et d'élargir son champ d'utilisation.

En définitive, ce travail a permis de concevoir et de réaliser une application utile, structurée et cohérente, répondant aux besoins du suivi doctoral. Il représente une expérience enrichissante qui a mobilisé à la fois des compétences en analyse, en conception, en développement web et en validation logicielle. Le portail de suivi du doctorat constitue ainsi une base sérieuse pour des améliorations futures et une éventuelle mise en production.

Bibliographie

[1] « Postman documentation overview », Postman Docs. Consulté le: 31 décembre 2025. [En ligne]. Disponible sur: https://learning.postman.com/docs/introduction/overview/

[2] « Lesson: JDBC Basics (The Java™ Tutorials > JDBC Database Access) ». Consulté le: 3 janvier 2026. [En ligne]. Disponible sur: https://docs.oracle.com/javase/tutorial/jdbc/basics/index.html

[3] « Docker: Accelerated Container Application Development ». Consulté le: 7 mai 2026. [En ligne]. Disponible sur: https://www.docker.com/

[4] M. Benabdellah, M. Derraz, et M. Legssyer, « Travaux collaboratifs de recherche doctorale basés sur l'e-Learning », *Doss. Rech. En Économie Gest.*, vol. 2, n° 2, p. 137-148, juill. 2013, doi: 10.34874/IMIST.PRSM/doreg-v2i2.15273.

[5] X. Schepler, « Solutions globales d'optimisation robuste pour la gestion dynamique de terminaux à conteneurs », Theses, Université du Havre, 2015. Consulté le: 8 mai 2026. [En ligne]. Disponible sur: https://theses.hal.science/tel-01251966

[6] « Welcome to Apache Maven – Maven ». Consulté le: 8 mai 2026. [En ligne]. Disponible sur: https://maven.apache.org/

[7] « Spring Framework Documentation :: Spring Framework ». Consulté le: 8 mai 2026. [En ligne]. Disponible sur: https://docs.spring.io/spring-framework/reference/index.html

Annexe A : Autres Interfaces Graphiques

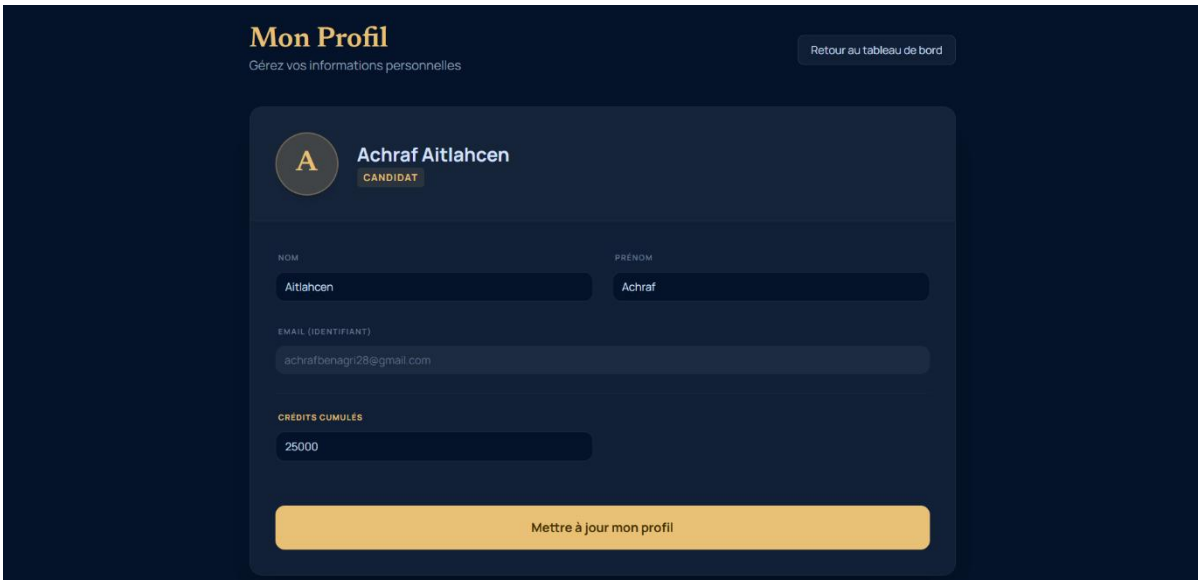


Figure 22 : Profil doctorant

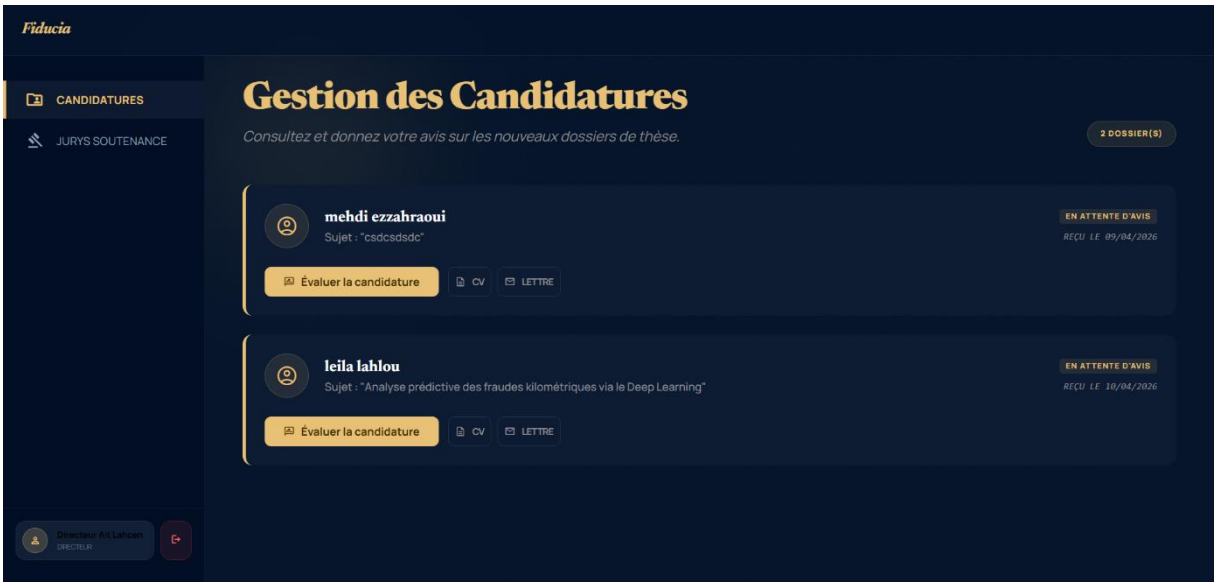


Figure 23 : Dashboard directeur gestion des candidatures

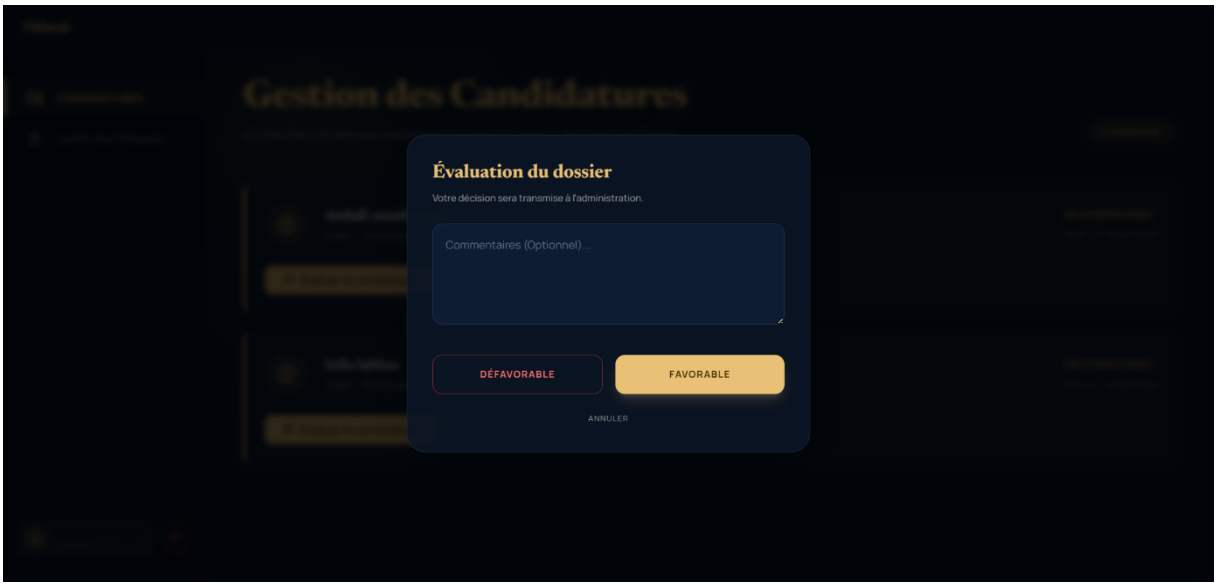


Figure 24 : Evaluation directeur



Figure 25 : Proposition jury par directeur

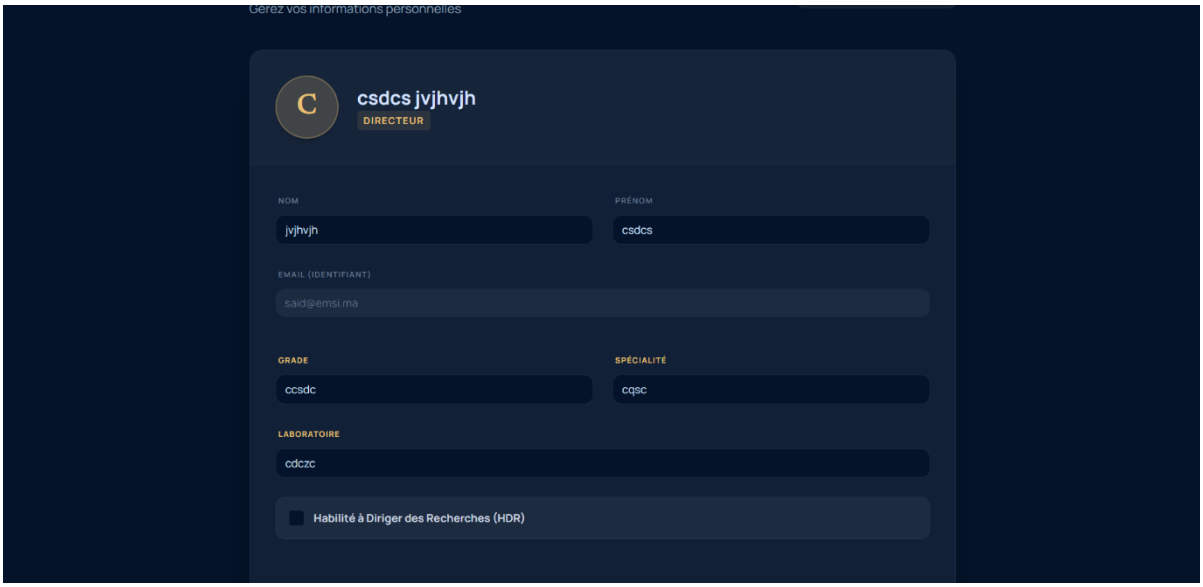


Figure 26 : Profil directeur

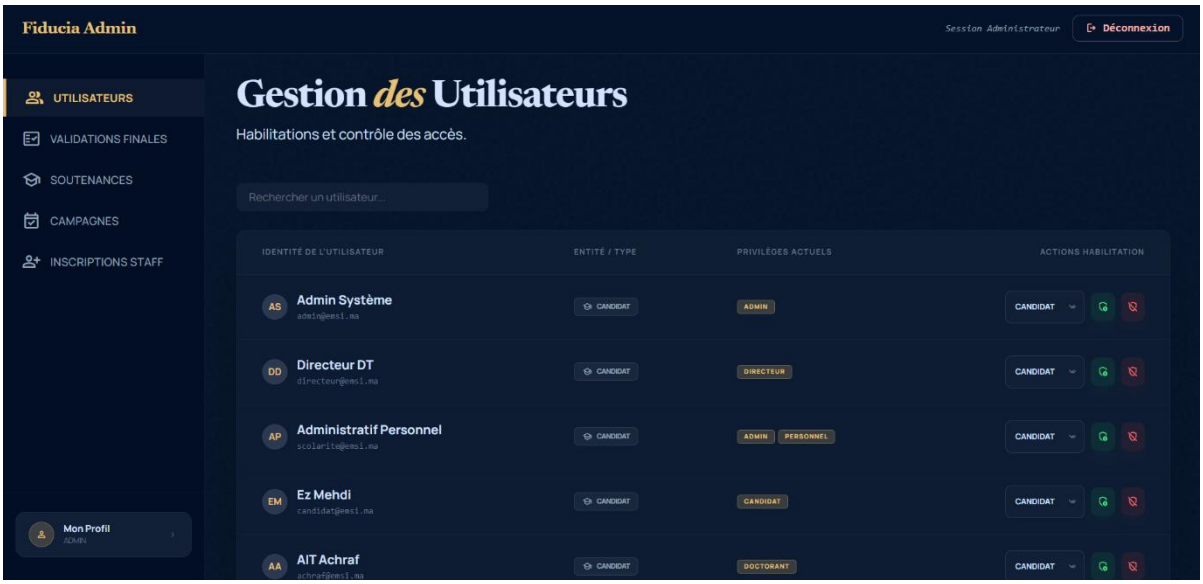


Figure 27 : Gestion des utilisateurs

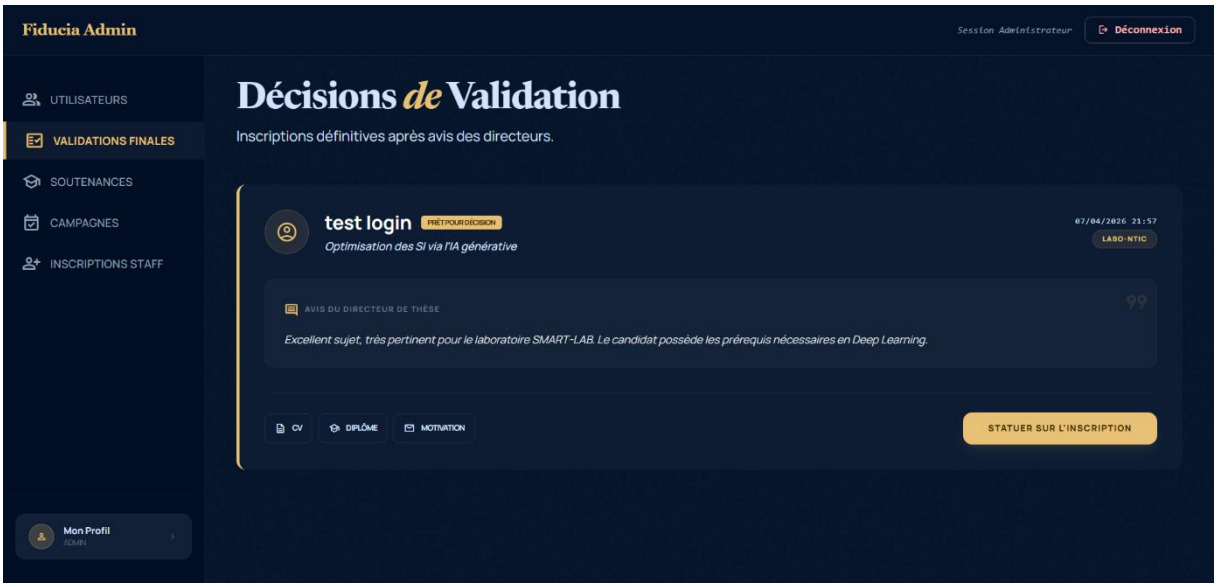


Figure 28 : Validation dossier par admin

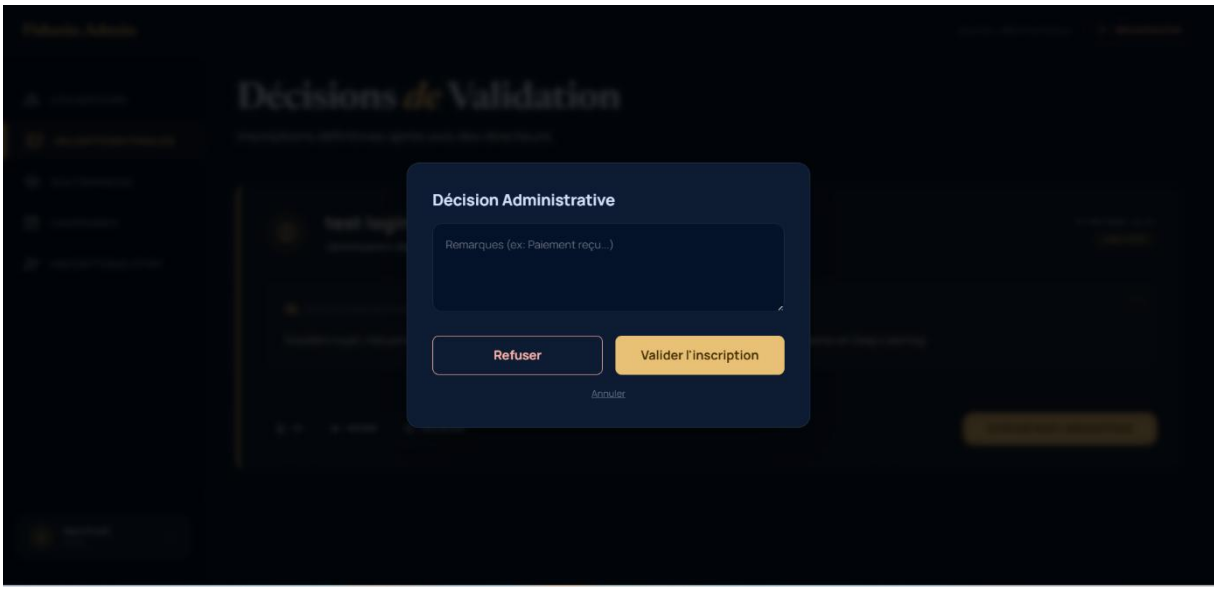


Figure 29 : Décision admin

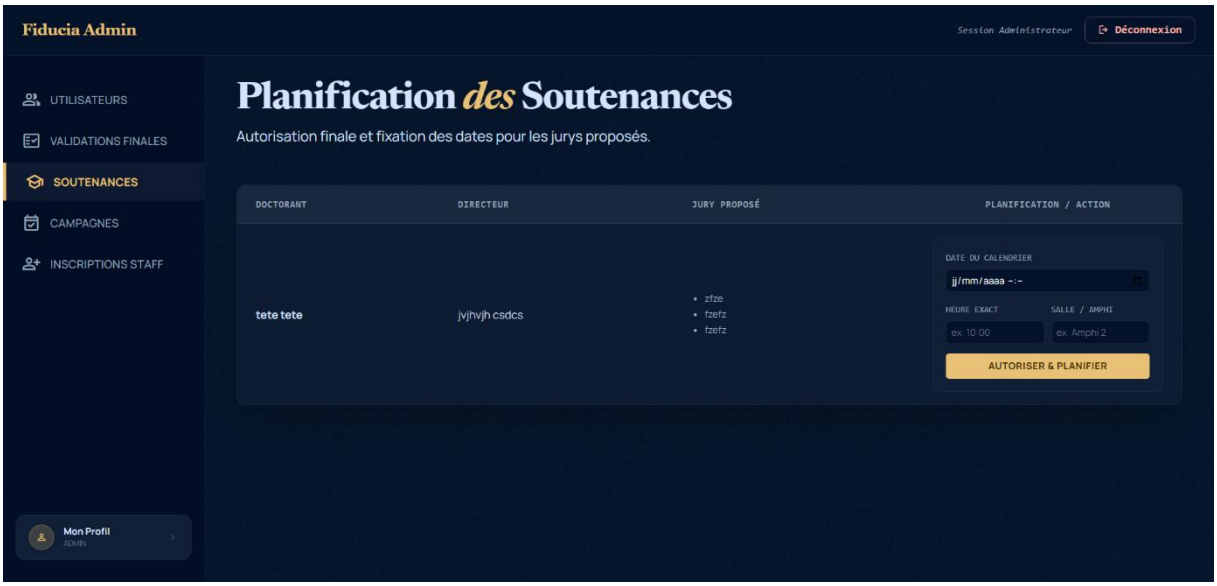


Figure 30 : Planification de soutenance par admin

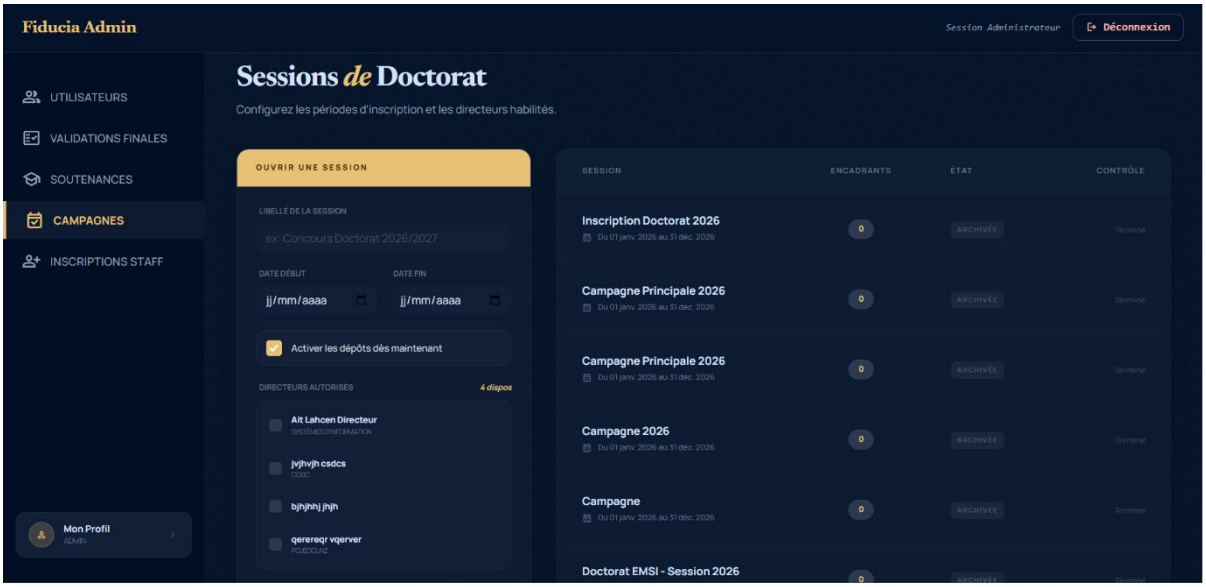


Figure 31 : Gestion des campagnes d'inscription

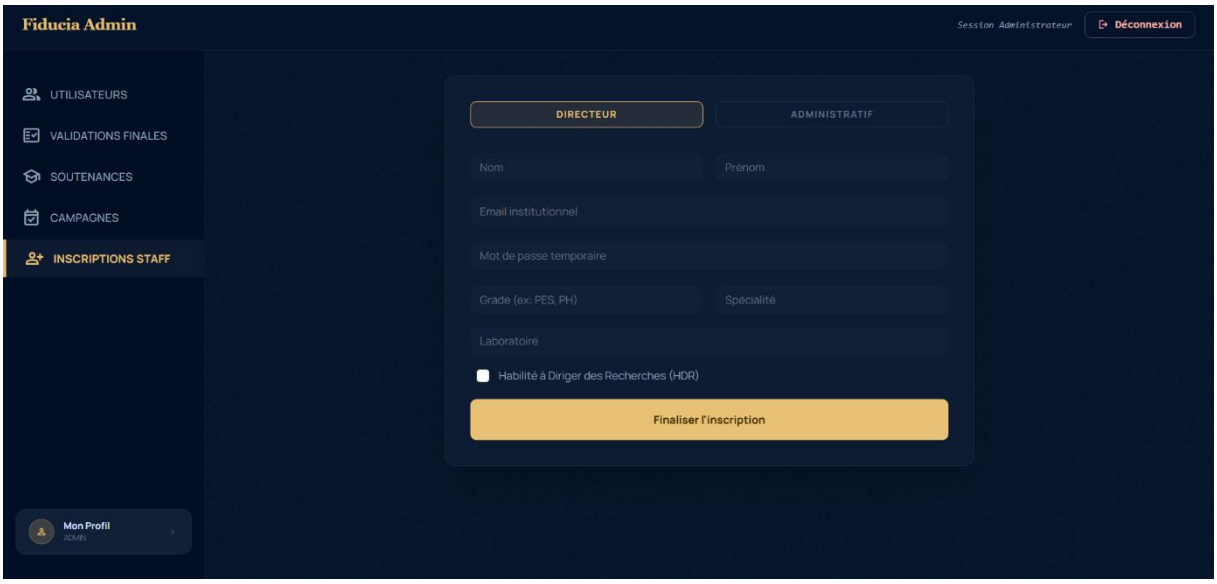


Figure 32 : Inscription des directeurs

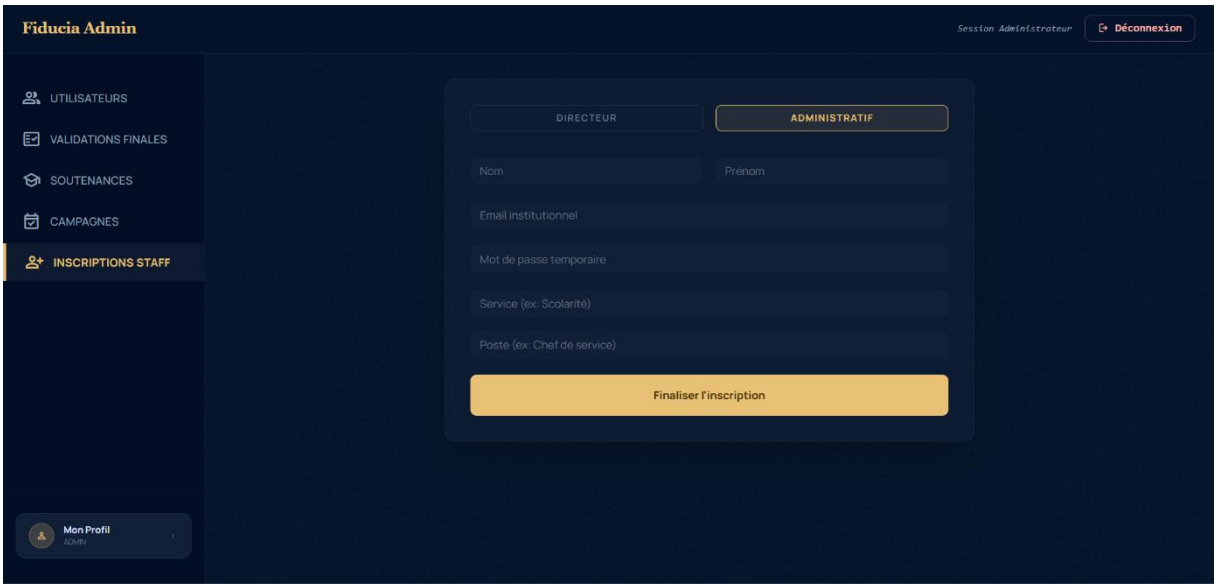


Figure 33 : Inscription administrative

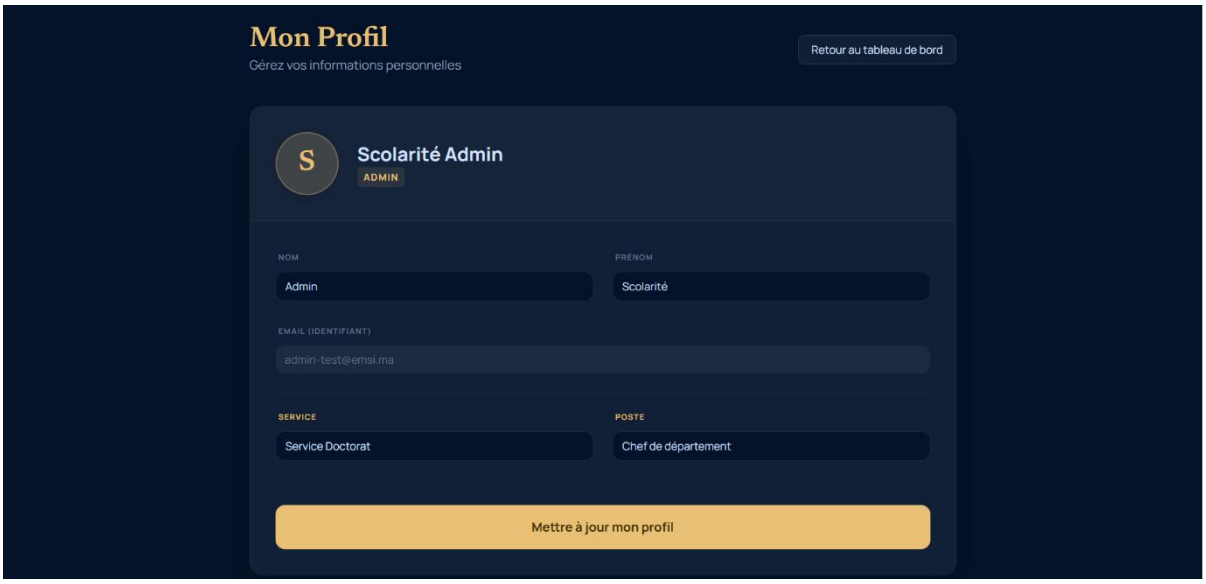


Figure 34 : Profil administration



Figure 35 : Résultat soutenance